Analysis of univariate forecasts on financial assets returns with time series models

Leonardo Diaz

August 2023

Abstract

The following study analyzes the use of statistical and machine-learning time series models to produce forecasts on asset returns. An extensive literature review of the topic is presented. ARIMA, GARCH and random forests models are implemented to predict and their accuracy is compared. The findings point out some of the differences in the prediction intervals obtained from each model, helping to establish key differences between ARIMA and GARCH forecasts. The point forecasts are similar in these models but that can change depending on how the point forecast is established. This is further explained in the discussion section. Finally, we realized that when it comes to point forecasts the machine learning random forests appear to be superior than the time series statistical models.

Contents

1 Introduction **3**

2 Literature Review **3**

3 Theoretical framework **9**

3.1 Homoskedasticity models 9

3.2 Heteroskedasticity models 13

3.3 Residual Analysis 15

3.4 Random forests 16

4 Data **20**

5 Methodology **23**
5.1 ARIMA model fittings 23
5.2 GARCH model fittings 28
5.3 GARCH model validation plots 33
5.4 Random forest model fittings 35

6 Results **36**
6.1 ARIMA models 36
6.2 GARCH models 40
6.3 Random forests models 45

7 Discussion **48**

8 Conclusions **49**

References **50**

9 Appendix **53**
9.1 Code 1: Fit random forests 53
9.2 Code 2: Fit univariate time series GARCH 60
9.3 Code 3: Fit univariate time series ARIMA 67

1 Introduction

Forecasting financial asset returns has always been a strenuous task where accuracy is vital. Different methods and techniques have been taken that try to effectively achieve this. The approach that we will consider in this study is via time series data. We will go over past literature on the topic and then introduce the technical information to be able to create our own models. Afterwards, we talk about the data used and the methodology implemented for the fitting of the models. Finally, results are shown and analyzed in the discussion and conclusion. The aim of this study is to analyze the effectiveness of univariate time series models when it comes to predicting where the future returns of assets will lie, we consider the exact point forecast predictions as well as interval predictions. We realize that it is most convenient to take into account both types of predictions. We analyzed four data sets in total. Two come from cryptocurrencies, one from an index and the last one from a stock. Two types of statistical time series models were implemented and also a machine learning algorithm. In the end we compare the results obtained and conclude if these types of univariate models give utility on the forecast of the returns of financial assets, what is the uniqueness we obtain and which of the three models manages to provide the most useful information.

2 Literature Review

In recent years several papers have emerged that have tried to forecast returns on the emergent cryptocurrency market with daily prices as the main dependent variable. The approach to do this is most commonly via the log returns. We will start by talking about a paper by Catania et al. (2019) where the authors use univariate and multivariate models for point and density forecasting on cryptocurrency. The data they use for their models are the closing log returns on the cryptocurrencies (and lags of these) with a set of crypto-predictors which include stock index prices, interest rates and treasury rates. One of the reasons they include these predictors is due to the analyzed correlation of bitcoin and the stock market during uncertainty periods. It has been shown that the shocks from the stock market have influenced bitcoin's volatility during the pandemic and other periods of turmoil (Nguyen, 2022). Another reason is the fast spreading market sentiment among different type of assets (Bianchi, 2018). For a volatility proxy they use the difference between the high and low price of each day. Over 7 univariate models are considered and a lot of combinations are available but they rely on Dynamic Model Averaging (DMA), a methodology introduced by (Raftery, Kárný, & Ettler, 2010) to find the optimal models. Around 18 multivariate models are tested

and benchmarked, which include constant parameter VARs and the time-varying parameter VAR proposed by (Koop & Korobilis, 2013). It is important to note that for point forecasts they use the Mean Squared Error (MSE) of each of the forecast horizons considered and for density forecasts they use the predictive log score. For their univariate forecast results, DMA does well with success rates higher than 60% for some cryptocurrencies at one day horizons and above 50% in most cases (Catania, Grassi, & Ravazzolo, 2018). For the multivariate forecast results they were surprised as only one model (a standard VAR) provided statistically superior point forecasts at long horizons when compared to the univariate models. They realize that in both types of models the crypto-explicatives and financial predictors do improve the forecast accuracy which shows us that the striking correlation of cryptocurrency with worldwide assets might be worth taking into account when doing forecast models to improve success rates. The success rates of the multivariate models are over 50% in all cases with some of 60% at two and three day ahead horizons. Their initial objective was to compare several alternative univariate and multivariate models to predict cryptocurrencies which they managed to do precisely. Their results highlight the importance of using model combinations (in both univariate and multivariate models) to achieve statistical significance and improve predictability gains. An important conclusion they mention is that new predictors based on crypto-market sentiment might be considered that could result in better point forecasts (Catania et al., 2018).

Another important recent addition to the literature is the study made by Vo and Slepaczuk (2022) where the key objective is to analyze the efficiency of forecasts from simple Autoregressive integrated moving average (ARIMA) models and hybrid ones (consisting of combinations with Generalized autoregressive conditional heteroskedasticity (GARCH) models) in algorithmic trading strategies focusing on the S&P500 index log returns from the start of 2000 to the end of 2019. For the ARIMA model fit they used a rolling window approach with different combinations of p and q where the model with the lowest Akaike information criteria (AIC) was used to generate the one day forecast return. After this, they tested a dynamic ARIMA-SGARCH model to forecast the returns in a similar manner. They considered trading strategy criteria to make an algorithm that goes long if the return will be positive the next day or short if it will be negative. For the forecast metrics they consider the Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), MSE and Mean Absolute Percentage Error (MAPE). Since it is a study on algorithmic investing they also consider performance statistics which include the annualized return compounded (ARC), annualized standard deviation (ASD) and the maximum drawdown (MD). Their results show that the hybrid

model outperforms the ARIMA and the buy and hold strategy based on the error metrics and performance statistics which shows that the hybrid model is indeed efficient for an algoritmic trading strategy. Some limitations they highlight are that the proposed ugarchroll function in R supports only moving one period ahead which reduces efficiency, their study did not include transaction costs, and maybe taking the consideration of the magnitude of the return instead of the sign might give better results (Vo & Slepaczuk, 2022).
NOTA: checar aqui si poner coma antes de esto o no en instead of the sign
ARIMA models have also been used to forecast return values on the Bombay Stock Exchange (BSE). Siva et al. (2018) try to predict returns on each stock of the BSE Sensex index by fitting an optimal ARIMA model to past data of each of the 30 companies that compose it. The data they use for their model are the actual returns, they use a proxy for volatility taking into account the difference from the maximum and the minimum of daily returns. After validating the stationarity of their data, they found the best fit of several models based on the lowest AIC. To measure the forecast of their accuracy they use the RMSE and MAE. They conclude that predicting with ARIMA models can be accurate when the forecast is short term, but may be uncertain on the long term (Latha, Nageswararao & Venkataramanaiah, 2018). It is important to note that some of the articles analyzed that use only ARIMA models to predict stock returns conclude that if they are efficient and accurate it is on a short term basis only. Ariyo et al. (2014) on their article *Stock price prediction using the ARIMA model* show by forecasting on Nokia stock price and Zenith Bank Index with experimental results that these models can compete well with emerging forecasting techniques in short-term prediction therefore guiding investors to make more profitable investment decisions (Ariyo & Olaniyan, 2014).

A combination of machine learning and ARIMA models to predict on cryptocurrency prices has also been tested by Saad Ali (2018). By using daily, weekly and monthly time series the author fitted ARIMA models to predict on three major cryptocurrencies: Bitcoin, Ethereum and Ripple. A partition of 80% training set and 20% test set was used. Using as metrics the MAE, MSE and RMSE the models done showed that ARIMA outperforms many other methods that try to predict cryptocurrency prices even though it is noted by the author that other data inputs as economic factors or number of transactions might improve the models (Alahmari, 2020).

Sebastiao and Godinho (2021) made a compilation of some of the papers that use log returns to predict in their paper *Forecasting and trading cryptocurrencies with machine*

learning under changing market conditions. They enumerated some that have tried to do this with different techniques including machine learning (ML) applications and have achieved it successfully. We will talk about the methods, data, metrics used and results achieved over some of these papers. First of all, Sebastiao and Godinho (2021) examined the predictability of returns with machine learning techniques. Their framework considered several models, linear, random forests and support vector machines. The predictability was analyzed as a regression problem and as a classification problem. The criterion used to get the best model was the maximization of the out of sample average return instead of the minimization of the forecast errors. Due to results in past literature they report that model averaging provides good outcomes for forecasting returns in cryptocurrency markets. They construct trading strategies that have win rates always over 50%, with the best ones achieved for Ethereum at 60.71%, they conclude that some of their strategies are able to significantly beat the market. The belief that ML techniques have potential in this market is supported even when there is a bear market. The aim of their study was to figure out if ML could potentially lead to profitable strategies in the cryptocurrency market and if profitability conditions exist even when the market conditions are constantly changing and a great amount of realistic features (for example transaction costs) are included. A lot of extra considerations like high frequency data, wider input sets considering technical analysis indicators or even bitcoin futures might lead to better results they claim (Sebastião & Godinho, 2021).

The literature of machine learning forecast in the cryptocurrency market, as noted by the previous authors, is extensive but another relevant approach is the one taken by Alessandretti et al. (2018) where they use an ensemble of regression trees to test whether lack of efficiency in the cryptocurrency market can be exploited to generate profits. An important metric used in their study is the return on investment (ROI) after one day which they define as:

$$ROI(c, t_i) = \frac{price(c, t_i) - price(c, t_{i-1})}{price(c, t_{i-1})}$$

Where c stands for the profitability of a currency. Two of their forecasting algorithms rely on XGBoost which is a scalable machine learning system for tree boosting and the third one is based on the long short-term memory (LSTM) algorithm for recurrent neural networks (Alessandretti, ElBahrawy, Aiello, & Baronchelli, 2018). They exclude bitcoin from the analysis as they consider cryptocurrencies prices expressed in terms of bitcoin. First they begin with the parameter setting which is the choosing of parameters specific to each method by looking for the ones that maximize either the ge-

ometric mean or the Sharpe ratio and set the amount of currencies per portfolio. Their best performing method is the LSTM which still achieves gains even when expensive transaction gains are considered. The approach taken was to build investment portfolios based on the predictions of each method and compare their performance with the moving average (MA) strategy. The three of their methods, they conclude, perform better than the MA strategy over the whole period considered. With their findings they conclude that algorithmic mechanisms can help anticipate the short-term evolution of cryptocurrency. Some limitations they mention are that they didn't consider prices from different exchanges and they only considered an average daily price instead of intraday price fluctuations. All of these could be considered to further improve their models they claim.

The prediction accuracy between univariate and multivariate machine learning models for forecasting price returns on cryptocurrencies is noticeable and important. Miller and Kim (2021) utilize several different models to show this. Their work focuses on predicting the log returns of the top 10 cryptocurrencies based on market capitalization. In terms of errors, they conclude that the multivariate LSTM networks perform better than the univariate machine learning and statistical time series models. They compared complex machine learning models to statisical models like ARIMA and exponential smoothing for forecasting. This was done in the univariate case but in the multivariate case instead of using the vector autoregressive time series models due to difficulties in computing the covariance matrices they used a multivariate LSTM machine learning model. For the forecast evaluation they used the RMSE and the MAD. Their conclusions also state that maybe considering GARCH or multivariate GARCH models might lead to better results since they account for volatility and adjusting the model parameters might improve the accuracy of the forecasts (Miller & Kim, 2021).

Accurately forecasting on the cryptocurrency market is a challenge not only because it is a small market by capitalization, compared to the NYSE its capitalization was only over 1/25 of it on June 2022 (Statista.com, 2022), but because it is also known for having a very high volatility while being susceptible to speculative bubbles (Katsiampa, 2017). Katsiampa (2017) found that the best model for bitcoin's volatility is the AR-CGARCH model where it is important to include a short and long run component on the conditional variance. The question in her paper *Volatility estimation for Bitcoin: A comparison of GARCH models* was which conditional heteroskedasticity model could best explain bitcoin data. To find this she analyzed daily closing prices of bitcoin, with which she then calculated the log returns of these. The research was done with

an autoregressive (AR) model for the conditional mean (of the returns) and a first-order GARCH for the conditional variance. To find the optimal model she used three information criteria AIC, BIC and HQ where the selected model is the one with the minimum values. After confirming there were indeed ARCH effects in the returns it was concluded that the AR model should be expanded to include an ARCH model for the variance. For her data the optimal model appeared to be an AR(1)-cGARCH(1,1). She concluded that since bitcoin is mainly an investment analyzing its volatility is important and this model appeared to be an adequate fit (Katsiampa, 2017).

Extensive research has been done considering time varying volatility for predicting cryptocurrency returns. For example, Stolte (2021) attempts to predict log returns using Bayesian stochastic volatility models with autoregressive components. The data consists of three cryptocurrencies which include bitcoin, litecoin and ethereum. The predictive power of multiple economic variables like the S&P500 index, the US bond rates and the price of gold futures is also analyzed and included in the models. The models used include a stochastic volatility VARX model, stochastic volatility VARX model with fat tails and a Markov Switching ARX model. They conclude that there are volatility regimes within the dynamics of the cryptocurrencies. Also, the predictor variables affect the return series differently across each regime they claim and note that some studies have shown that cryptocurrency return predictions are significantly more accurate with machine learning models than with statistical time series ones (Stolte, 2021). Limitations they find include that price data is limited, the cryptocurrency market remains influenced by manipulative participants and the methodology via simulation with Bayesian models is time expensive.

Having done all this research we believe important to individually analyze univariate model forecasts and see the benefit they can provide and how accurate they can be. It is clear that considering solely univariate input for the models will not be as rich as multivariate models would be but an important aim of this study is to highlight the importance of the models, comprehend their predictive power and see how far they can go in terms of efficient and useful forecasting.

3 Theoretical framework

3.1 Homoskedasticity models

A homoskedasticity model is one where the variances of the errors remain constant through time (Knaub, 2007). In this study we rely initially on ARMA models for forecasting. Box and Jenkins deeply explain this general linear process in their book *Time Series Analysis* (Box, Jenkins, Reinsel, & Ljung, 2016) and we will make use of this literature to explain it here again and have the necessary background to understand how our models are fitted and why they work.

Box and Jenkins begin by discussing the representation of a stochastic process as the output from a linear filter where the input is a white noise e_t and it has the form:

$$\widetilde{z}_t = e_t + \phi_1 e_{t-1} + \phi_2 e_{t-2} + ... = e_t + \sum_{j=1}^{\infty} \phi_j e_{t-j} \tag{1}$$

for stationary processes, $\widetilde{z}_t$ is the deviation from some origin or its mean value.

They state that equation 1 allows us to represent $\widetilde{z}_t$ as a weighted sum of present and past values of the white noise process e_t. The white noise can be seen as a series of shocks that drive the system, it is important to note that it is also a sequence of uncorrelated random variables with mean zero and a constant variance. If we want $\widetilde{z}_t$ to be a valid stationary process then its coefficients ϕ_j need to satisfy $\sum_{i=0}^{\infty} |\phi_j| < \infty$ (Box et al., 2016).

After the explanation of equation 1, Box and Jenkins introduce the Autoregressive process (AR) and the Moving Average process (MA). First, the AR process consists of a model where the first p of the lags of $\widetilde{y}_t$ weights are non zero:

$$\widetilde{y}_t = \psi_1 \widetilde{y}_{t-1} + \psi_2 \widetilde{y}_{t-2} + ... + \psi_p \widetilde{y}_{t-p} + e_t \tag{2}$$

The ψ_j are the weight parameters (given to each past observation at lag p) which now are a finite amount and consequently this process is called an AR of order p denoted as AR(p).

Next, the MA process consists of a model where only the first q weights of the white

noise are non zero:

$$\widetilde{y}_t = e_t - \theta_1 \widetilde{e}_{t-1} - \theta_2 \widetilde{e}_{t-2} - ... - \theta_q \widetilde{e}_{t-q} \tag{3}$$

This process is called a MA of order q denoted as MA(q). Where each θ_j is the weight given to past white noise at lag q.

As stated by Box and Jenkins, to achieve parsimony on the models used it might be necessary to include both autoregressive and moving average terms. So the mixed autoregressive-moving average process of order (p,q) denoted as ARMA(p,q) is (Box et al., 2016):

$$\widetilde{z}_t = \phi_1 \widetilde{z}_{t-1} + \phi_2 \widetilde{z}_{t-2} + ... + \phi_p \widetilde{z}_{t-p} + e_t - \theta_1 e_{t-1} - \theta_2 e_{t-2} - ... - \theta_q e_{t-q} \tag{4}$$

To talk about the stationary and invertibility conditions for this process it is important to show the definition provided by Box and Jenkins of the backward shift operator B which is the following: $By_t = y_{t-1}$ which implies $B^j yt = y_{t-j}$. Now, the stationary and invertibility conditions for this process are analyzed via the AR part and the MA part respectively. The stationary condition is important when it comes to time series analysis because these type of time series are the ones we fit into the Autoregressive processes (without having to difference the data until we find stationarity) which makes it easier to analyze in comparison to the non stationary ones with their statistical properties changing over time.
We will go over this later but most of the time series data in this paper is indeed stationary and that is because the daily log returns in most cases when it comes to stock prices are as they don't usually show any trends or seasonality patterns. Due to logarithmic properties, using this function on the returns is, in some sense, a way of differencing the data. This can be seen through the logarithm property:

$$\log(\frac{a_t}{a_{t-1}}) = \log(a_t) - \log(a_{t-1})$$

The stationary condition implies, and therefore guarantees, that the process will have a constant variance. With the backward shift operator we can see that equation 2 is equivalent to:

$$(\phi_1 B - \phi_2 B^2 - ... - \phi_p B^p)\widetilde{z}_t = \phi(B)\widetilde{z}_t = e_t \tag{5}$$

$\phi(B) = 0$ is the characteristic equation for the process. The stationary condition for the AR process consists in all the roots of this equation lying outside of the unit circle. Box and Jenkins note that this roots are the reciprocals to the roots of the polynomial equation in x:

$$x^p - \phi_1 x^{p-1} - \phi_2 x^{p-2} - ... - \phi_p \tag{6}$$

So for an AR process to be stationary all the roots of the previous polynomial equation must lie in the unit circle. That is if x_i is a root of the polynomial then $|x_i| < 1$.

Now, another important condition is for the MA part to be invertible. With the backward shift operator we have that the moving average process is equivalent to:

$$(1 - \theta_1 B - \theta_2 B^2 - ... - \theta_q B^q)e_t = \theta(B)e_t \tag{7}$$

The process is invertible if the roots of $\theta(B) = 0$ lie outside the unit circle (Box et al., 2016).
If both of this conditions are satisfied then the ARMA(p,q) process is stationary and invertible. We know from past related literature that the importance of the invertibility condition lies in its implication that the model can be used for forecasting as it allows us to assume that the correct generating formula is known for estimating the e_t's from the past values of the observed $\widetilde{z}_t$'s (Granger & Andersen, 1978).

Now, this process can be generalized when the time series data needs differencing an we have an autoregressive integrated moving average model (ARIMA). This model works for time series data that is not initially stationary but by differencing the intention is to make it. Before we show this process lets consider the differencing operator as $\nabla = 1 - B$ where B is the backward shift operator. Now,

$$\varphi(B)z_t = \phi(B)\nabla^d z_t = \theta_0 + \theta(B)e_t \tag{8}$$

where

$$\phi(B) = 1 - \phi_1 B - \phi_2 B^2 - ... - \phi_p B^p,\ \theta(B) = 1 - \theta_1 B - \theta_2 B^2 - ... - \theta_q B^q$$

is the general form of an ARIMA process of order (p,d,q) or also written as ARIMA(p,d,q). When d = 0 the model represents a stationary process, because $\phi(B)$ is assumed to be stationary and $\theta(B)$ invertible (Box et al., 2016).

These models are important for our research because we will use them to forecast the daily log returns of the assets analyzed. Box and Jenkins claim that forecasts are calculated most simply by directly using the difference equation as this is most convenient from a pure computational standpoint (Box et al., 2016). That is how the forecast function in R manages to forecast on ARIMA models.

An important thing to note is that to forecast we must first find the best fit for the time series data we are considering. To do this, Box and Jenkins came up with a methodology (identification and estimation procedure) that consists in the following. First, the stationarity of the process y_t must be assessed, and if it does not satisfy it, one should difference y_t until stationarity is achieved. Second, identify the resulting ARMA model for the differenced or not differenced process. To assess this the sample autocorrelation function (ACF) and sample partial autocorrelation function (PACF) need to be analyzed. The key points to look at are the following:

- Failure of the estimated ACF to die out exponentially might suggest that we should treat the process as nonstationary in y_t but it might be stationary in a higher order of difference. Taking this into account, the degree of differencing d which is required to achieve stationarity is reached when the ACF of $v_t = \nabla^d y_t$ dies out rapidly. An important note Box and Jenkins make is that once stationarity is achieved further differencing should be avoided to stop the introduction of serial correlation.

- Now for the p and q parameters. If both the ACF and the PACF tail off a mixed process is suggested (AR and MA). First it should be noted that while the ACF of an AR process of order p tails off, its PACF has a cutoff after lag p. Opposite of this we have that the ACF of a MA of order q has a cutoff after lag q, while its PACF tails off. The authors state that in general, AR (MA) behaviour, as measured by the ACF, tends to mimic MA (AR) behavior as measured by the PACF. Therefore, the analysis behind this is that the ACF of an AR(p) decays exponentially, while the PACF cuts off after the lag p. Likewise, for a an MA(q) where its ACF cuts after lag q, its PACF decays exponentially. This shows that the lags where the ACF and PACF cut below the significance level can give us an initial good estimate for q and p respectively (Box et al., 2016).

An important thing to mention is model selection criteria. The AIC and BIC order selection procedures are useful but Box and Jenkins claim they should be viewed as supplementary tools to assist in the selection process. This measures should aid the

careful examination of the estimations of ACF and PACF of the time series. In our research we considered the AIC information criteria to verify which model might be best. The AIC information criteria is defined as:

$$AIC_{p,q} = \frac{-2\ln(ML)+2r}{n}$$

Where ML is the maximized likelihood of estimating a range of potential models and r = p + q + 1.
For the purposes of this study we will use the previous mentioned approach when looking for the best fit to our data of an ARIMA model and compare the models by the AIC scores they produce.

3.2 Heteroskedasticity models

Contrary to the previous models, an heteroskedasticity model is one where the variance of the errors differ. These type of models are more often considered in cases of linear regression. An idea that puts this importance into perspective is that in most cases little error values are more logically assumed to have smaller variances and larger error values greater variances (Knaub, 2007).

We will introduce the ARCH model and its generalization the GARCH model. The autoregressive conditional heteroskedastic (ARCH) model was originally proposed by Engle (1982) to try to assess the conjecture that the unpredictability of inflation was a primary cause of business cycles. The uncertainty due to this unpredictability directly affected investors' behavior, so Engle needed a model where this uncertainty changed over time (Tsay, 2010). In a stationary ARMA process we have that the unconditional mean of the series is constant over time, while the conditional mean varies as a function of past observations. This is analogous in the ARCH model, but the difference lies in that even though it takes the unconditional variance of the error process over time as constant, it allows the conditional variance of ϵ_t to vary as a function of past squared errors (Box et al., 2016) (here we talk of ϵ_t as the error term in an AR process which in equation 2 we had written as e_t).

To see this, let F_t be the information set available until time t, which can be thought of as the σ-field generated by ϵ_{t-j}, $j \geq 1$ (Tsay, 2010). Then the ARCH model of ϵ_t has the following properties: E$\{\epsilon_t|F_{t-1}\}$ = 0 and var$\{\epsilon_t|F_{t-1}\}$ = σ_t^2 where this is the conditional variance of ϵ_t given the past F_{t-1} information. The basic ARCH(s) model

can be seen as:

$$\epsilon_t = \sigma_t a_t \tag{9}$$

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + ... + \alpha_q \epsilon_{t-q}^2 \tag{10}$$

where $\{a_t\}$ is a sequence of iid random variables that have mean zero and variance equal to one. With $\alpha_0 > 0, \alpha_i \geq 0$, for $i = 1, ..., q-1$, and $\alpha_q > 0$.
As explained by Box and Jenkins, the parameter constraints are imposed in this way to ensure that the conditional variance is positive. For modelling purposes, the a_t are usually assumed to follow either a standard normal, or a student t distribution (Box et al., 2016).
Now, the Generalized ARCH model (GARCH), proposed independently by Bollerslev (1986) and by Taylor (1986) (Tsay, 2010), assumes the conditional variance to be a linear function of its own lags, and has the form:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{q} \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2 \tag{11}$$

where $\alpha_0 > 0, \alpha_i \geq 0$, for $i = 1, ..., q-1$, $\alpha_q > 0$, $\beta_j \geq 0$, $j = 1, ..., p-1$ and $\beta_p > 0$. Similarly to the ARCH model, the constraints are sufficient for the conditional variance to be positive.
Both of these models allow for volatility clustering, and are be able to capture thick tailed behavior of the underlying unconditional distributions. It is important to note that in symmetric ARCH and GARCH models the variance depends on the magnitude of the effects perceived but not on the signs (Box et al., 2016).
The use of these models will allow us to forecast volatility on the time series data analyzed, and to see how these predictions compare to the actual data forecasts coming from the mentioned ARIMA models.

Another important model is the hybrid ARIMA-GARCH model. As we have explained, stock prices can be really volatile at times of recessions or bull markets. With this in mind, if we have an ARIMA model for stock prices (or returns) the homoskedastic assumption might be violated because the variance on the errors doesn't behave as constant (which happens really often in real life scenarios), so when we have this, the errors are said to be heteroskedastic and we have to consider ARCH effects.

The hybrid ARIMA(p,d,q)-GARCH(r,s) model can simultaneously predict the conditional mean and the conditional heteroskedasticity of the process (Vo & Slepaczuk,

2022), it has the following form:

$$
\begin{gathered}
y_t = \phi_1 y_{t-1} + \phi_2 y_{t-2} + ... + \phi_p y_{t-p} + \epsilon_t - \theta_1 \epsilon_{t-1} - ....\theta_q \epsilon_{t-q} \\
\sigma_t^2 = \alpha_0 + \sum_{i=1}^{r} \alpha_i \epsilon_{i-1}^2 + \sum_{j=1}^{s} \beta_j \sigma_{t-j}^2 \\
\epsilon_t = z_t \sigma_t \\
\epsilon_t \sim N(0, \sigma_t^2)
\end{gathered}
\tag{12}
$$

Where the parameter specifications and conditions are all the ones stated above on the previous models.

3.3 Residual Analysis

It is also important for our study that we mention three hypothesis tests performed on time series models and their residuals to check for the adequacy of fits and the presence of ARCH effects. The following three tests will be used in our study:

- Ljung-Box test
- Weighted Ljung-Box test
- Engle's Lagrange multiplier (LM) test for ARCH

Ljung-Box test

The Ljung-Box test consists in analyzing the residuals of a given ARMA fit and their autocorrelations. By analyzing the quantity:

$$
\widetilde{Q}(r) = n(n+2) \sum_{k=1}^{m} \frac{r_k^2}{n-k} \tag{13}
$$

where $r_k = \sum_{i=k+1}^{n} a_t a_{t-k} / \sum_{t=1}^{n} a_t^2$ and $(a_1, ..., at)$ are the residuals.

If this is distributed as a χ^2_{m-p-q} for a large n (where the parameters are known) then the model is deemed appropriate yielding an approximate test for lack of fit (G. Ljung & Box, 1978).

Weighted Ljung-Box test

This test is similar to the previous one with a difference that consists in attributing a

weight of $\frac{m-k+1}{m}$ to the inside of the sum in the $\widetilde{Q}$ resulting in the following test statistic: $\widetilde{Q}(r) = n(n+2)\sum_{k=1}^{m} \frac{m-k+1}{m} \frac{r_k^2}{n-k}$, defined this way in Fisher and Gallagher (2012). It is important to mention that the results using this weighted test have shown a higher efficacy in simulation experiments (Fisher, 2012).

Engle's ARCH LM test

Before estimating an ARCH model it is desirable to verify whether it is appropriate for the data being analyzed. According to Engle (1984), the LM test procedure is ideal having become one of the most popular tests in practice (Hong & Shehadeh, 1999). The idea of the test is the following, it begins with running an ordinary least squares (OLS) regression and saving the residuals. Afterwards, regressing the squared residuals on a constant and p lags. Finally, testing the test statistic as an asymptotically distributed χ_p^2, if this holds, the null hypothesis is said to be true which claims that there are no ARCH effects (Engle, 1982). We purposefully left out the derivation of the test statistic as it exceeds the technicality of this study but we believe that this explanation is enough to move forward and apply the test having shown the main idea behind it.

3.4 Random forests

Before we introduce and explain the concept of random forests we will talk about supervised learning. This is simply a formalization of the idea of learning from examples, the main objective is for the learner to "learn" from certain labeled examples in the training set so that it can identify unlabeled examples in the test set with the best accuracy it can manage to achieve (Learned-Miller, 2014). More formally, the training set consists of m ordered pairs $(x_1, y_1), (x_2, y_2), ..., (x_m, y_m)$, where x_i can be a set of measurements of a data point and y_i is the label for the data point. The test data is another set of k measurements that come without labels. The objective of this type of learning is to make educated guesses about the most appropriate labels for the test data. Now, since the objective of this study is to analyze the forecasting of time series, we have to take into account that the order of the data is crucial. We need data from the past to predict data from the future, so the sets have to be ordered. As described by Brownlee (2020) one way of doing this is to convert the time series sets into a supervised learning problem where the measure x_i consists of the data until time i, and the label y_i consists of the data in time $i+1$. This way the learner will try to identify the labels of the test set in an ordered way with the information from the past labels which are essentially the previous data points.

Having understood this, we will now talk about decision trees to be able to explain random forests. For this, we need to use the following definitions of trees described by Louppe (2015):

- A tree is a graph in which any two vertices are connected by exactly one path.
- A rooted tree is a tree in which one of the nodes has been designated as the root. It can be thought of as a directed graph (all edges are directed away from the root).
- Terminal nodes (also known as leaves) are those with no edges going out of them.

Taking this into account, a decision tree can be defined as a model $\phi : \mathcal{X} \rightarrow \mathcal{Y}$ represented by a rooted tree where any node t represents a subspace $(\mathcal{X}_t)$ of the input space with the root node corresponding to $\mathcal{X}$ itself. The internal nodes of the tree are labeled with a split taken from a set of questions. The split divides the subspace $\mathcal{X}_t$ into two subspaces that are mutually exclusive. The terminal nodes have a best guess value of the output variable. For this study we are considering regression trees so the best guess $\widetilde{y}_t \in \mathcal{R}$ and the predicted output is the mean of the leaf reached by the instance after being propagated by the splits (Louppe, 2015).

Now, random forests, as stated by Breiman (2001), are a mixture of tree predictors where each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest. This ensemble is then allowed, in a classification random forest, to vote for the most popular class (Breiman, 2001). To grow these mixtures of trees (random forests), random vectors are generated which control the growth of each tree in the ensemble.
An important example to do the latter is the bagging technique where to grow each decision tree a random selection with replacement is made from the observations in the training set. This is a general aggregation scheme which generates bootstrap samples from the original data set, which in the case of a regression random forest, constructs a predictor from each sample and decides by averaging (Biau & Scornet, 2016). A bootstrap sample consists in choosing n times from n points with replacement to compute the individual tree estimates. Another popular technique originated from Breiman´s original proposal is the CART-split criterion, which consists in selecting, at each node of each tree, the best cut by optimizing the CART-split criterion which is based on the so-called Gini impurity (for classification) or the prediction squared error (for regression). For more on this, see *A random forest guided tour* by Gérard Biau and Erwan

Scornet section 2.3.

All the techniques to grow the trees have in common that for the kth tree, a random vector θ_k is generated, independent of the past generated $\theta_k's$ but with the same distribution, and a tree is grown using the training set and θ_k which results in a classifier $r(x, \theta_k)$ where x is an input vector.

In bagging, the random vector θ is generated as the counts in S boxes resulting from S darts thrown at random at the boxes, where S is the number of observations in the training set. After a large number of trees has been generated they each vote for the most popular class and this is what the random forest technique consists of. Formally defined by Breiman: A classification random forest is a classifier consisting of a collection of tree-structured classifiers $r(x, \theta_k), k = 1, ..$ where the θ_k are independent identically distributed random vectors and each tree casts a unit vote for the most popular class at input x (Breiman, 2001).

Breiman also shows that random forests do not overfit as more trees are added but instead produce a limiting value of the generalization error showing that random forests converge (Breiman, 2001). In the following paragraphs his explanation can be seen.

Given an ensemble of classifiers $h_1(x), h_2(x)..., h_K(x)$, with the training set drawn at random from the distribution of the random vector (X,Y), Breiman defines the margin function as:

$$mg(X, Y) = av_k I(h_k(X) = Y) - max_{j \neq Y} av_k I(h_k(X) = j)$$

with I the indicator function. This margin function measures the extent to which the average number of votes at (X,Y) for the right class exceeds the average vote for any other class. The larger the margin, the more confidence in the classification given by the technique. The generalization error is given by:

$$PE = P_{X,Y}(mg(X, Y) < 0)$$

where the the probability calculated is over the (X,Y) space.

In random forests we have that $h_k(X) = h(X, \theta_k)$. Now, for a large number of trees it follows from the Strong Law of Large Numbers and the tree structure that as the

number of trees increase, for almost surely all sequences $(\theta_1, \theta_2, ...)$ the generalization error converges to $P_{X,Y}(P_\Theta(h(X,\Theta) = Y) - max_{j \neq Y} P_\Theta(h(X,\Theta) = j) < 0)$.

This last proof is shown by Breiman and explains why the random forests do not overfit as more trees are added but indeed produce a limiting value of the generalization error (Breiman, 2001). The previous result shows why pruning is not necessary in random forests (which use bagging) and a higher depth of trees instead helps solve the potential overfitting issue.

To find bounds for the generalization error in random forests, he proceeds to define the margin function in a random forest as:

$$mr(X,Y) = P_\Theta(h(X,\Theta) = Y) - max_{j \neq Y} P_\Theta(h(X,\Theta) = j)$$

He then defines the raw margin function as:

$$rmg(\Theta, X, Y) = I(h(X,\Theta) = Y) - I(h(X,\Theta) = j(X,Y))$$

Where $j(X,Y) = argmax_{j \neq Y} P_\Theta(h(X,\Theta) = j)$.

He then goes on to show an upper bound for the generalization error which is given by:

$$PE \leq \hat{\rho}(1 - s^2)/s^2$$

where $\hat{\rho}$ is the mean value of the correlation between two raw margin functions with different Θ values and s is the strength of the set of classifiers $\{h(x,\Theta)\}$ defined as

$$s = E_{X,Y} mr(X,Y).$$

Breiman explains that the bound is likely to be loose but it still shows that the two ingredients involved in the generalization error for random forests are the strength of the individual classifiers in the forest, and the correlation between them in terms of the raw margin functions.

It is important to note that the random forests do not use the popular resampling method of cross validation, and it would not be of use in this study due to the assumption, in cross validation, that the observations are independent from each other. Since we are dealing with time series data it is clear our observations lack independence, hence why

the bagging technique is used for the creation of random forests.

The algorithm for the construction of the random forest proposed by Breiman can be explained following Biau and Scornet's summary in their previously mentioned paper. They mention that in Breiman's forests each node of a single tree is associated with a hyperrectangular cell. The root of the tree is the space $\mathcal{X}$ and, at each step of the tree construction, a node (its corresponding cell) is split in two parts. Afterwards, the terminal nodes (leaves) altogether form a partition of $\mathcal{X}$. The algorithm works by growing M different randomized trees. This is done in the following way. Before each tree is constructed, a_n observations are drawn at random with replacement from the original data set. Now, only these observations are taken into account in the tree building. Then, at each cell of each tree, a split is performed by maximizing the CART-criterion over *mtry* directions chosen uniformly at random among the p original ones, where *mtry* represents the number of possible directions for splitting at each node of each tree. Finally, construction of individual trees is finished when each cell contains less than *nodesize* points where *nodesize* is chosen to be between 1 and the number of observations chosen (a_n). For any query point $x \in \mathcal{X}$, each regression tree predicts the average Y_i (among the a_n points) for which the corresponding X_i falls into the cell of x (Biau & Scornet, 2016). Notably, growing the tree and obtaining an estimation only depends on the preselected data points (a_n's). For more detail on the CART-criterion see (Breiman, 1984) and (Biau & Scornet, 2016).

4 Data

The data considered for this study consists of past prices of two cryptocurrencies: Bitcoin and ADA, one index: S&P500 and one stock: Ford. On all the models we will use the daily log-returns (obtained from the daily closing prices) as the dependent variable due to their important statistical properties. In financial mathematics, simple stock returns are commonly assumed to follow a log normal distribution which implies that log returns follow a normal distribution and this allows us to use the properties of the normal distribution easily on the data. This is helpful because they can be added across time periods to create cumulative returns, the distribution is only dependent on the mean and standard deviation of the data. The log returns are calculated by the following formula:

$$lr_t = \ln(\frac{C_t}{C_{t-1}})$$

where C_t stands for the current closing price (at time t) and C_{t-1} for the previous day closing price (at time t-1). It is evident that if we have n observations of closing prices then we will have $n-1$ daily log returns for each day but the first one. An important relationship in financial mathematics on stock closing prices and returns mentioned above is the following (Miskolczi, 2017):

$$\ln(\frac{C_t}{C_0}) = \ln(C_t) - \ln(C_0) = \ln(1 + r_t)$$

where r_t is the simple return.
It is important to note that since the cryptocurrency market is open 24 hours a day for every day of the week. There isn't a closing price per its definition but the data considers it as the price at midnight of each day. This is different in the S&P500 and Ford data we have where they are indeed closing market prices. Another thing to note is that for cryptocurrency we have information for every day of the year, while the information on S&P500 and Ford exclude the days when the market is closed, so the data sets, even though they are obtained from the same four years, are considerably smaller.

The following tables provides a summary of descriptive statistics for the daily log returns of all the assets.

Table 1: Summary Statistics for Bitcoin and Ada log returns

	Train	**Test**	**Overall sample**
Start date	8-July-2018	19-Sep-2021	8-July-2018
End date	18-Sep-2021	8-July-2022	8-July-2022
Observations	1168	293	1461
Bitcoin			
Mean	0.00168	-0.00272	0.000797
SD	0.039	0.0363	0.0385
Median	0.00139	-0.00111	0.00127
Min	-0.465	-0.174	-0.0465
Max	0.172	0.136	0.172
Kurtosis	19.2	2.48	16.5
Skew	-1.39	-0.361	-1.21
Ada			
Mean	0.00239	-0.00554	0.000799
SD	0.0591	0.0521	0.0578
Median	0.00173	-0.00488	0.00061
Min	-0.504	-0.204	-0.504
Max	0.279	0.17	0.279
Kurtosis	6.44	1.38	5.81
Skew	-0.319	-0.0815	-0.269

Table 2: Summary Statistics for S&P 500 and Ford log returns

	Train	**Test**	**Overall sample**
Start date	8-July-2018	18-Sep-2021	8-July-2018
End date	17-Sep-2021	8-July-2022	8-July-2022
Observations	805	202	1007
S&P500			
Mean	0.000578	-0.000635	0.000334
SD	0.0142	0.0137	0.0141
Median	0.00108	-0.0000677	0.001
Min	-0.128	-0.0412	-0.128
Max	0.0897	0.0301	0.0897
Kurtosis	17.7	0.248	14.6
Skew	-1.04	-0.412	-0.924
Ford			
Mean	0.000377	-0.000678	0.000165
SD	0.0261	0.0313	0.0272
Median	0	-0.00193	-0.000846
Min	-0.117	-0.102	-0.117
Max	0.211	0.11	0.211
Kurtosis	7.9	1.18	5.86
Skew	0.487	0.189	0.394

All the data used in this study was obtained from Yahoo finance (*Finance Home*, n.d.). The data gathered from yahoo finance of both cryptocurrencies (Bitcoin and ADA) come from coinmarketcap.com (CMC) as they started a partnership where CMC provides the data to yahoo.

5 Methodology

5.1 ARIMA model fittings

The first approach consisted in fitting a univariate ARIMA model for the log return predictions. For each asset we fitted two models on the training sets, one with the auto.arima function in R which finds the best fit by looking for the optimal parameters of an ARIMA model given the data set by minimizing the fit's AIC score (among other considerations) and the other one was fitted according to an own analysis using the Box Jenkins Methodology by analyzing the partial auto correlation plot (PACF), auto correlation plot (ACF) and the standardized residuals. We used the training/test division methodology to measure the accuracy of our predictions on the test set having

fitted our models with the training sets.

First, for the bitcoin training data it was concluded by the auto.arima function that the best fit was an ARIMA(1,0,1) model. Afterwards, our own analysis began with performing an augmented Dickey Fuller test on the observations which helped us conclude that the data was indeed stationary so the time series didn't need any differencing and we could set the d parameter to 0. Then we looked at the ACF plot and saw that the first two lags were outside of the significance level so a parameter of q = 2 seemed appropriate. The PACF plot showed that also the first two lags were outside of the significance level so a parameter of p = 2 was deemed appropriate. With this we had our second model which consisted of an ARIMA(2,0,2). The following are the ACF plot and the PACF plot of the bitcoin training data:

Figure 1: **ACF plot Bitcoin**

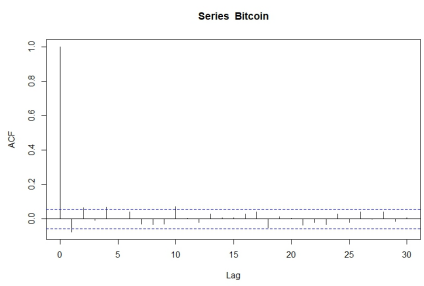
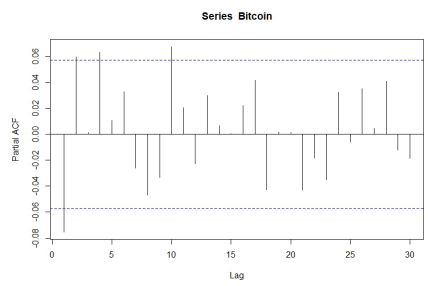


Figure 2: **PACF plot Bitcoin**



The same approach was used for the other three data sets. For the S&P500 training data the best fit by auto.arima was an ARIMA(4,0,1) and using the Box Jenkins Methodology to find the parameters we considered ARIMA(4,0,2) a good fit, since the PACF plot shows that the first two lags are outside the significance level, the third one is close to the significance level and the fourth one is outside p was set to 4, d was set to 0 because similarly to the bitcoin data set an ADF test was performed which showed the stationarity of the data. Lastly, q was set to 2 because the ACF shows the first two lags outside the significance level and the third one inside but far away from it. An important thing to note is that the ARIMA(4,0,1) fit had a larger AIC score (-4690.14) than the ARIMA(4,0,2) (-4697.58) which according to the theory proposed by Box and Jenkins makes it a worse model since lower AIC scores indicate better fits. The following are the plots analyzed:

Now for the ADA training data set a difference was spotted. This was that the auto.arima

Figure 3: **ACF plot SP500**

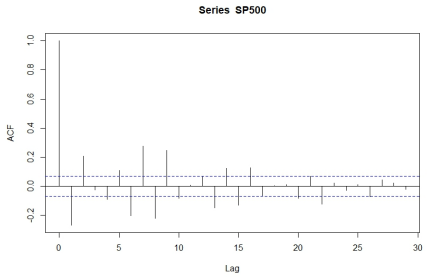
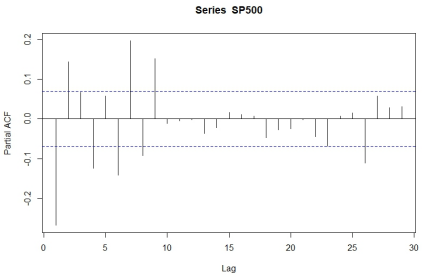


Figure 4: **PACF plot SP500**



function found the best fit to be an ARIMA(5,1,0) which due to the d=1 parameter it claims that the original training data is not stationary so the model should be differenced. On the other hand, with the Box Jenkins methodology we found the following things. First the PACF plot showed the initial two lags outside of the significance level which indicate a good value for the parameter p as 2, then the ACF the same way showed the first two lags outside the significance level and the third one really far away so q was chosen as 2, finally the ADF test showed that the null hypothesis was rejected indicating that the training time series is stationary so an ARIMA(2,0,2) was deemed appropriate. Analyzing both AIC scores we can see that the first model has one of -3129.47 and the second one of -3299.97 which indicates a better fit. It is important to know that we still used both to forecast to analyze the different results.

Figure 5: **ACF plot ADA**

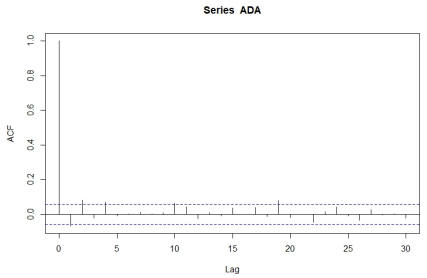
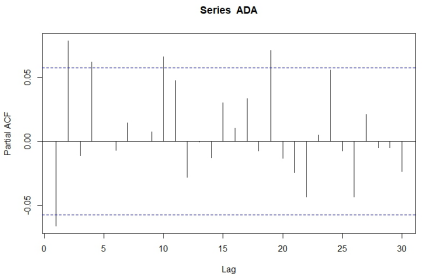


Figure 6: **PACF plot ADA**



Finally, for the Ford data set the auto.arima function found the best fit to be an ARIMA(0,0,0) which indicates a white noise process. This shows the randomness of stock returns. Manually analyzing and using the Box Jenkins Methodology it was seen that the parameters are the same, the ACF plot and PACF after lag 0 are way below the signif-

icance level, the augmented dickey fuller test claims stationary so an ARIMA(0,0,0) does seem to be a good fit. Here we show the following plots for the Ford training data set:

Figure 7: **ACF plot Ford**

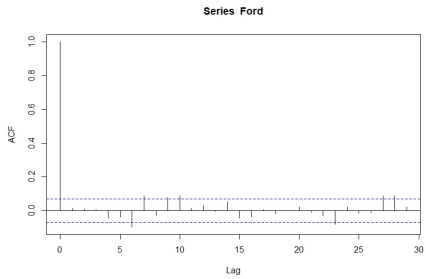
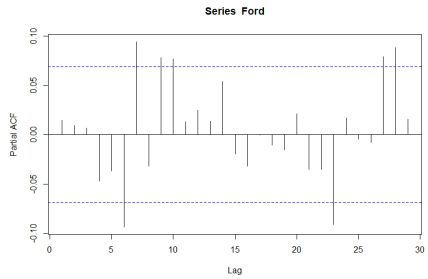


Figure 8: **PACF plot Ford**



Before forecasting it is important to check if the models are adequate and if they are we should proceed with the predictions. To verify this, one thing we can do (as explained in the theoretical framework) is analyze the residuals of the fittings. To do this we can perform a Ljung-Box test on them, if the p-value is above the significance level then it means there is strong evidence for the null hypothesis of the test which states that the residuals are most likely independently distributed and we can reject the alternative hypothesis which states that they exhibit correlation. If the residuals are independently distributed then the model doesn't show a significant lack of fit and we can proceed with using it for forecasting.

It is important to note that the choice of parameters for the Ljung-Box test, especially the choice of h (the amount of lags being tested), affects the outcomes of it and its power therefore an appropriate choice of it has to be done. Across the literature there have been different suggestions on choosing this value. As explained by (Hassani & Yeganegi, 2019) several authors like (G. M. Ljung, 1986), (Tsay, 2010), (Shumway & Stoffer, 2010), (Hyndman & Athanasopoulos, 2021) give different empricial suggestions for h. These authors propose the optimal value of h to be 5, ln(T), 20 and min(10,T/5) respectively, where T is the number of observations in the time series.

Since the objective of the Ljung-Box test is model selection and adequacy checking, inaccurate results by selecting inappropriate values of h could affect the whole model building and implementation procedures (Hassani & Yeganegi, 2019), because of this

Hassani et al. (2020) look for a way of selecting an optimal lag order for the test. After performing a series of simulations to analyze the power of the test on different size time series with different values of h their conclusions show that the optimal value of h depends on T and on the test's level α (a conclusion that previous literature on the topic had ignored). When T is around 1000, their findings find appropriate a value for h of 50 when $\alpha = 0.05$ and 25 when $\alpha = 0.01$. Since our training set sizes are around that number (T=1168 for the cryptocurrency data and T=805 for the stock and index) we will use their results by considering their suggested optimal value of h when performing Ljung-Box tests. As noted on the theoretical framework of this study the degrees of freedom of the chi-squared distribution for the test should be set to h-p-q for every model's test.

The following table shows the Ljung-Box test results (p-values) on the different models with the two different significance levels α and the corresponding optimal h considered.

Table 3: Ljung-Box test results

Significance level	$\alpha = 0.05$	$\alpha = 0.01$
Bitcoin		
ARIMA(1,0,1)	0.1963	0.257
ARIMA(2,0,2)	0.1872	0.1972
ADA		
ARIMA(5,1,0)	3.526e-08	1.736e-10
ARIMA(2,0,2)	0.3968	0.3424
S&P500		
ARIMA(4,0,1)	0.007473	0.002356
ARIMA(4,0,2)	0.002597	0.009152
Ford		
ARIMA(0,0,0)	0.0007993	0.0142

We notice that for both bitcoin models and on both α levels the residuals are independent so the ARIMA models don't indicate a significant lack of fit. Therefore we will use both of these to forecast.

For the ADA models we can see that the first one has correlated residuals on both significance levels which tells us that the first ADA model is not adequate for prediction, so we won't use it. It is important to note here that residual correlation could mean many things, one that we will discuss later is that the time series might have ARCH

effects which would imply that the variance of the errors in the model is not really well defined as a constant so we should take it as a random variable and therefore use a better model to forecast like the ARCH or GARCH. The second ADA model has better results as in both levels the alternative hypothesis is rejected, the residuals appear to be independently distributed which shows that the model doesn't have a lack of fit and therefore we will use it for forecasting.

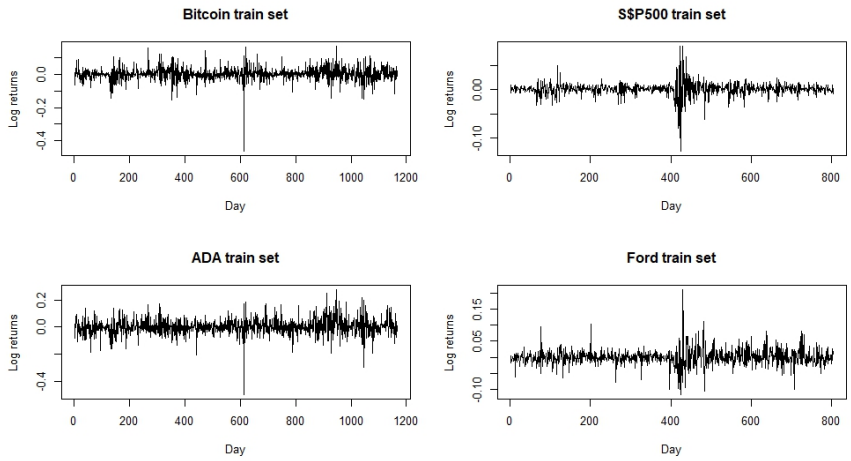
Now, for the S&P500 data we notice that on both models and on both levels the null hypothesis is always rejected. These results indicate that the models show a significant lack of fit which means that forecasting with these models is not optimal. Since the p-value of the second model (on the $\alpha = 0.01$ test) is close to the significance level I will use this model to show forecasts even though we shouldn't deem them as accurate but just to show the errors and see why the fit is considered inadequate. We can conclude from these findings that it is highly likely that a better model that takes into consideration changes in the error term's variance like ARCH or GARCH is more accurate for forecasting on the S&P500.

Finally, we can see that for the white noise model of the Ford data the residuals appear to be only independent on a 0.01 significance level so we will use this model also to predict.

Having justified the adequacy of the models we then used the forecast function from R to predict the next n values of the log returns of each asset where n is the size of the test set of each asset. The idea was to compare and see which models worked better and judge if an ARIMA univariate model is appropriate for predicting log returns. We will analyze these predictions in the results section.

5.2 GARCH model fittings

After doing the univariate ARIMA models we now considered fitting GARCH models to forecast the returns. The reason for this was because the variance is evidently not constant throughout every point in time for the log returns of the assets. We can see this in several ways and both reasons justify fitting these type of models. First, through a plot of the log returns of each asset we can see that there are spikes in different periods indicating high volatility at some points which appears to be varying through time.
We notice a really high spike in the returns, around the 430 for the index and the stock



and around the 600 day mark for the cryptocurrencies, the dates of these outliers in the returns coincide with the dates of the 2020 stock market crash due to the uncertainty of the COVID-19 pandemic.

Now, we can also analyze the squared residuals of the best ARIMA fittings for each asset to check for ARCH effects, if these are correlated then it implies that the time series exhibits conditional heteroskedasticity (Bollerslev, Engle, & Nelson, 1994) which tells us that ARIMA models might be insufficient to predict and we might consider a more robust model like GARCH, to do this we will perform a Ljung-Box test on the squared residuals. Another way to check for ARCH effects is to perform a Lagrange multiplier test like the one proposed by (Engle, 1982), where the null hypothesis is that the data has no ARCH effects. We applied both of these tests and the results are in the following table.

We used the same suggested amount of lags as in the previous analysis of the residuals but here we also considered the proposed h=ln(T) (Tsay, 2010) for the Ljung-Box test to see the different results and analyze them. The significance level on the following tests is always taken as $\alpha = 0.05$.

Table 4: Ljung-Box test on squared residuals (p-values)

Number of lags	h=50	h=$\ln T$
Bitcoin		
ARIMA(1,0,1)	0.9982	0.006786
ARIMA(2,0,2)	0.9919	0.0005459
ADA		
ARIMA(5,1,0)	6.939e-10	$<$ 2.2e-16
ARIMA(2,0,2)	0.2257	6.68e-08
S&P500		
ARIMA(4,0,1)	$<$ 2.2e-16	$<$ 2.2e-16
ARIMA(4,0,2)	$<$ 2.2e-16	$<$ 2.2e-16
Ford		
ARIMA(0,0,0)	$<$ 2.2e-16	$<$ 2.2e-16

Table 5: ARCH Lagrange Multiplier test on residuals (p-values)

Number of lags	h=50	h=$\ln T$
Bitcoin		
ARIMA(1,0,1)	0.9999	0.02921
ARIMA(2,0,2)	0.9996	0.01688
ADA		
ARIMA(5,1,0)	0.0007248	9.869e-14
ARIMA(2,0,2)	0.7797	0.0001025
S&P500		
ARIMA(4,0,1)	$<$ 2.2e-16	$<$ 2.2e-16
ARIMA(4,0,2)	$<$ 2.2e-16	$<$ 2.2e-16
Ford		
ARIMA(0,0,0)	2.645e-12	$<$ 2.2e-16

These findings are relevant. As we can see, when we analyze 50 lags for the Ljung-Box tests, the results show that the both Bitcoin fits and the second ADA fit have independent squared residuals while all the other fits for the assets appear to have correlated residuals. For the ARCH LM-test we can see the same conclusions, both Bitcoin fits and the second ADA fit don't appear to have ARCH effects while all the others do. Now, if we consider, as proposed by Tsay (2010), to analyze ln(T) lags for the tests we can see that all tests now conclude that the models present ARCH effects. The difference in these results clearly lies in the amount of lags analyzed for the test (as we had stated before). A reason for this could be that on the cryptocurrency time series

squared residuals when we analyze a smaller amount of lags ln(1168) = 7.06 they appear to be correlated but when we take into account a way bigger amount as 50 then they don't display any serial correlation. We believe this might be due to the correlation being present in smaller subsets of the squared residuals and not on large ones. In other words, it might be possible that when we consider a big amount of these they don't display significant correlation but more individually they do. Due to these findings we will consider and fit GARCH models on all the data sets and see how they forecast if they are deemed adequate fits because from the tables we can conclude that all models show ARCH effects.

For the fittings of the GARCH models we considered a function that iterated over 9 possible fits alternating the GARCH order (from 1 to 3 in each component) while keeping the ARMA order constant (the same one as the two fits considered on the ARIMAs for all data and the p=0, q=0 for the Ford data). So for Bitcoin, ADA and S&P500 we analyzed a total of 18 GARCH models for each data set and for Ford only 9 models were analyzed. We chose the models with the lowest AIC for each time series. The order specifications will be shown and also the goodness of each fit will be analyzed and we will conclude if the models are adequate fits so we can proceed with the forecasting or not.

For the fittings of the GARCH models we considered a function that iterated over different possible GARCH fits. For all models, the possible order went from 0 to 3 on both GARCH parameters while for the ARMA order the iteration was dependant on the best fit found from the previous section. So, for example, in the Bitcoin time series the best ARMA model found (and the one used to forecast) was the ARMA(2,2) so the function to find the best GARCH fit iterated both ARMA parameters from 0 to 2. This results in 105 GARCH fits for bitcoin that actually converge and are able to be analyzed. Out of all of these the one with the lowest AIC score was selected. Now, the models obtained (for each asset) had to be correctly validated to see if we could proceed with the forecasting. To do this we removed the parameters that were not significant. This will be shown and explained below. The function used and the models analyzed can be accessed through the code given in the appendix.
It is important to note, as done in past literature by (Katsiampa, 2017) and (Ruppert, n.d.), that to analyze the goodness and adequacy of a GARCH or ARMA-GARCH model fit we should look at the standardized residuals and the standardized squared residuals. If these exhibit any autocorrelation then the model is not necessarily a good fit and we shouldn't be to confident to predict on it. To do this we can look at the

GARCH model specifications after the fit and a series of Weighted Ljung-Box and ARCH LM tests are performed by the ugarchfit function in R which will let us conclude if the models are adequate, along with analyzing the significance of each individual parameter and removing them from the fit if not significant. We will present the p-values of the tests that come with the fittings after showing the chosen model fit to predict on each data set.

In all the following models, except for the SP500 one, we assumed the a_t in the error term that is multiplying σ_t (as explained in the theoretical framework in equation 9) to follow a t-student distribution as with this distribution the best AIC scores are obtained in the models. On the SP500 model we used a normal distribution for the error term as the QQ plot generated with the t-student assumption on the model validation was curved contrary to the one using a normal distribution.

Taking into consideration the lowest AIC score we chose the following models for each data set and then proceeded to their validation. For Bitcoin the best fit was an ARMA(0,2)-GARCH(1,1). For ADA an ARMA(10,0)-GARCH(1,1) with all the autoregressive coefficients from order 2 to 9 set as 0 (this is due to a spike in the autocorrelation plot of the standarized residuals at lag 10 for the model ARMA(1,0)-GARCH(1,1) showing us that the ar10 is significant). For the S&P500 an ARMA(1,0)-eGARCH(1,1). Finally, for Ford a GARCH(1,1) was deemed the best fit.

The following table shows the p-values of the tests performed on the standardized squared residuals (Weighted Ljung-Box) and on the residuals (ARCH LM).

Table 6: Weighted Ljung-Box and ARCH LM tests

Test	Bitcoin	ADA	S&P500	Ford
Weighted Ljung-Box				
Lag = 1	0.7928	0.5628	0.2661	0.3546
Lag = 5	0.8820	0.5085	0.3826	0.8364
ARCH LM				
Lag = 3	0.5801	0.6695	0.8661	0.6459
Lag = 5	0.6146	0.4507	0.1843	0.9024

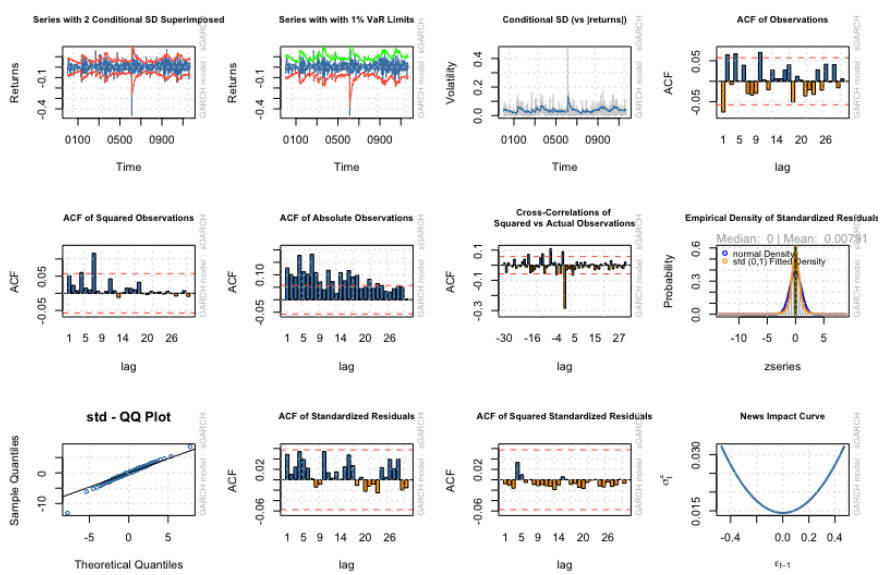
As we can see from the previous table all p-values on both tests (considering a different amount of lags) are above the significance level which means for the Weighted Ljung

Box test that the null hypothesis of the standarized squared residuals is not rejected therefore we can claim they are independently distributed. Similarly in the ARCH LM test the null hypothesis of the residuals not having ARCH effects is not rejected, therefore the residuals of the models do not present any more ARCH effects. These two tests for all models allow us to conclude that they don't present a significant lack of fit and it is adequate that we proceed with the validation.

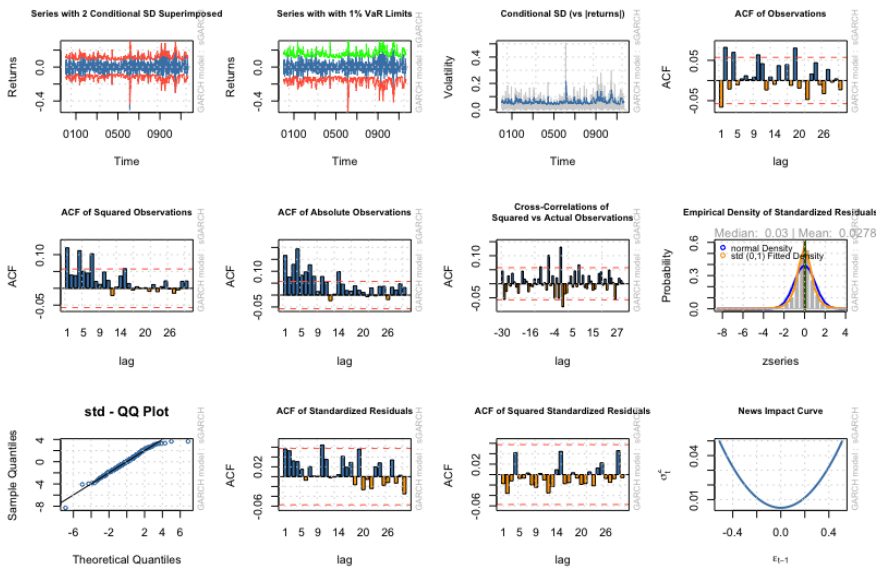
We also analyzed the significance of every term in the ARMA and the GARCH process removing the ones with p values above the threshold of 0.05. We can see the following plots that aided in each model validation in the next section.

5.3 GARCH model validation plots

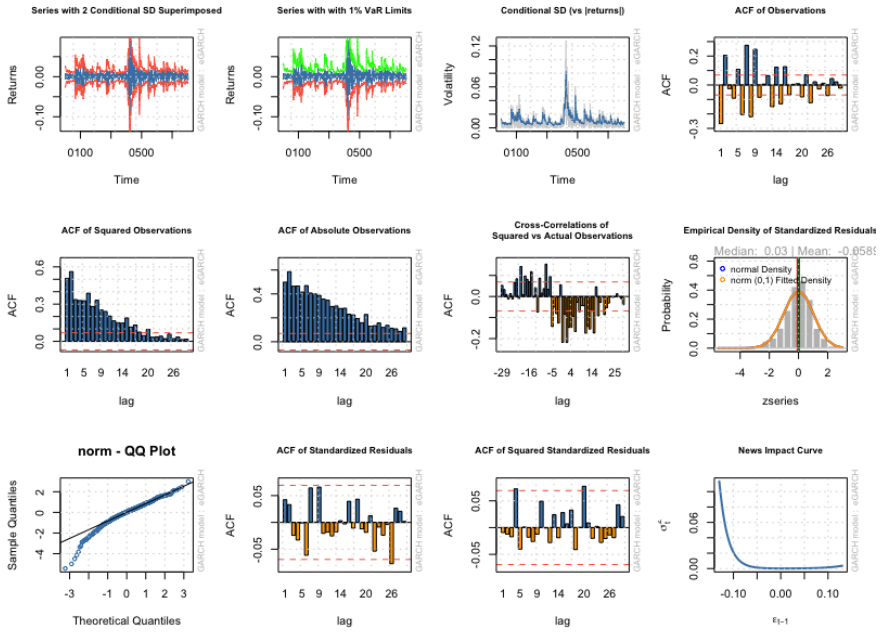
Bitcoin - ARMA(0,2)-GARCH(1,1):



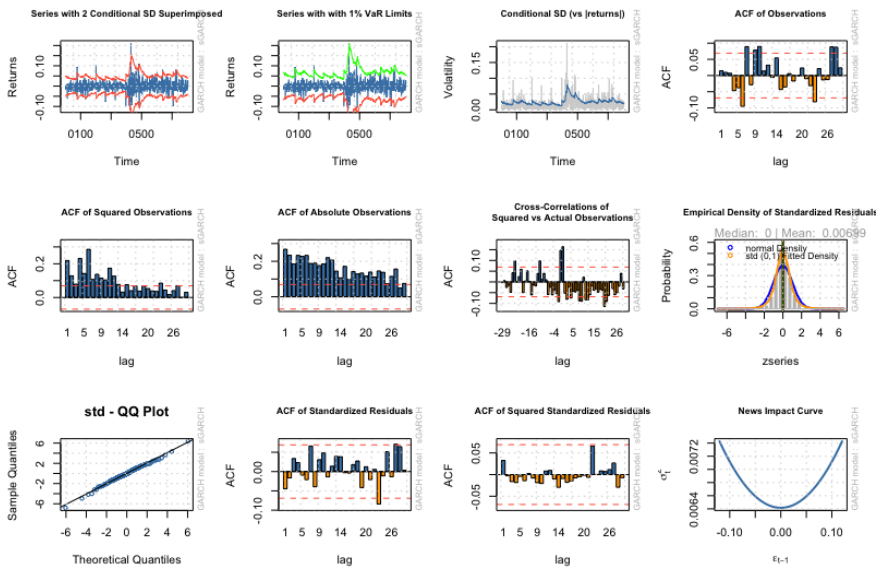
ADA - ARMA(10,0)-GARCH(1,1) with all the autoregressive coefficients from order 2 to 9 set as 0:



S&P500 - ARMA(1,0)-eGARCH(1,1):



Ford - GARCH(1,1):



Having gone through the model validation to predict we considered a rolling forecast where the predictions were done one step at a time rolling the forecast forward n times (where n is the size of the test set of each data set). This was done with the ugarchforecast function in R from the rugarch package which allows us to easily compute a conditional rolling forecast and an unconditional full test size forecast. Both of these results will be shown in the following section.

5.4 Random forest model fittings

Finally a univariate log return random forest was done to predict on the same data sets (training and test) and obtain point forecasts. For this we first turned the time series into a supervised learning dataset (as explained in the theoretical framework) considering 10 lags of the log returns as the measure for every ordered pair. When working with time series the validation technique is particularly important because it has to take into account the order of the data, so it is not appropriate to use the common k fold validation technique (as mentioned in the theoretical framework). Walk forward validation has been a popular technique in validating these type of models especially those predicting stock prices, in this type of validation the dataset is divided into parts chronologically ordered and in each go, all the data that is available before the time point to predict is used as the training set and the part to predict is used as the test set (Falessi, Huang,

Narayana, & Turhanc, 2020). Because of this, to create our random forests we used this validation technique where every point forecast was predicted taking into account all the previous test data. Our model used a total of 100 estimators (trees) for each data set analyzed. To create the random forest we used the RandomForestRegressor module from the sklearn.ensemble library in Python. As discussed by Breiman, a good value for the mtry parameter in a regression problem is $d/3$ (Scornet, 2020) where d is the number of predictors. In our case we have one predictor which are the log returns but since we are using 10 lags of it we considered using an mtry parameter of 10/3 or 3 (rounding down) and indeed got slightly better results than with the default value for mtry which in the RandomForestRegressor module is called max_features because as stated in the documentation this is the number of features to consider when looking for the best split. It is important to note that in classification problems a good value for mtry that Breiman mentions is the square root of d.
We will show the resulting prediction vs test graphs and the error at the end of the results section.

6 Results

6.1 ARIMA models

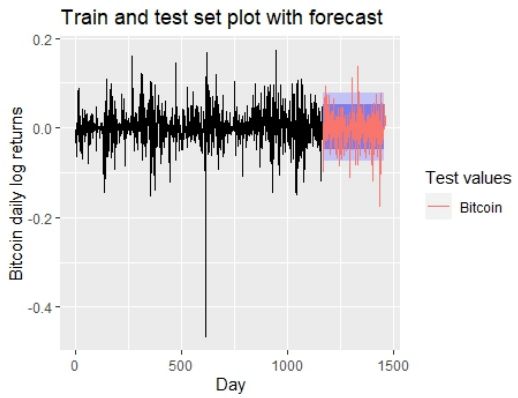
The metrics used to compare and measure the accuracy of the ARIMA point forecasts were the the mean absolute percentage error (MAPE), the root mean squared error (RMSE) and the normalized root mean squared error (NRMSE). To compute these errors we took the test data and analyzed it against the one step forecast which in these ARIMA models is the mean value of the prediction interval. It is very important to note, as mentioned by (Hyndman & Athanasopoulos, 2021), that if we only produce point forecasts we can't tell how accurate the predictions are hence the importance of also producing prediction intervals because they help assess the uncertainty associated with each forecast. Therefore, point forecasts have almost null value if they don't come along with prediction intervals. It is clear that the actual test data when analyzed as a point estimate will be significantly different in size than the mean of the prediction interval (point forecast) because this will be close to zero but overall the prediction intervals allow us to quantify this uncertainty. First, we will analyze the univariate ARIMA forecasts. For each asset we fitted two ARIMA models as explained in the methodology. Here we will show the plot of the forecast with the prediction intervals (PI) and the training and test data plotted against these and then show the errors of the point forecast compared to the test data, just to give us an idea of the point estimates of

our ARIMA models.

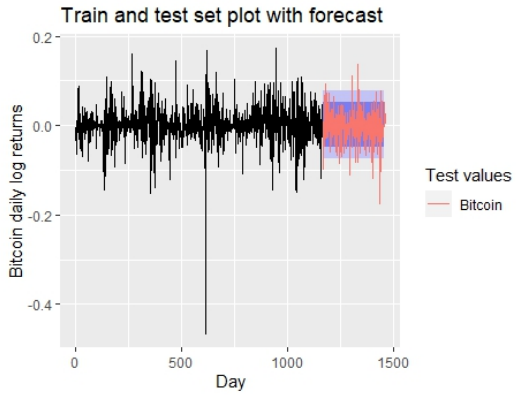
The following are the plot forecasts of Bitcoin returns:

Figure 9: **Bitcoin ARIMA model forecasts**

(a) ARIMA(1,0,1): PI: (80,95)



(b) ARIMA(2,0,2): PI: (80,95)



Model	MAPE	RMSE	NRMSE
ARIMA(1,0,1)	1.026254	0.036541	0.117945
ARIMA(2,0,2)	1.026832	0.036541	0.117954

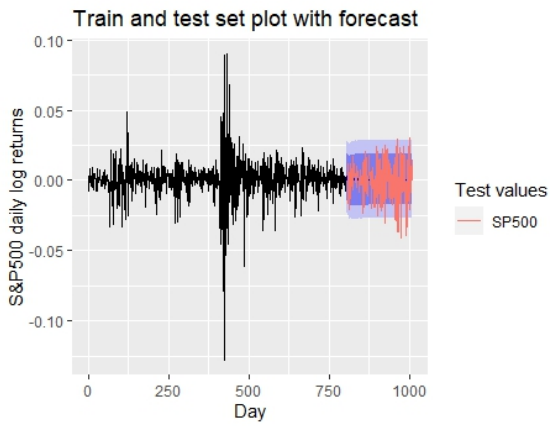
Now, we can see that the errors are similar to each other for both models and since the normalized root mean square error is low with an 11% value we can conclude this is not entirely a bad point estimate for the daily log returns of bitcoin. Overall it is clear that the first model has smaller errors so we can conclude that the ARIMA(1,0,1) model

is the best one for the bitcoin log returns and it is important to take into consideration when using the point forecasts the prediction interval as it accounts for the uncertainty of each prediction. We can see the value the prediction interval provides as most of the test data lies inside of it except for a small number of outliers.

For the S&P500 we have the following plot:

Figure 10: **S&P500 ARIMA model forecasts**

(a) ARIMA(4,0,2): PI: (80,95)



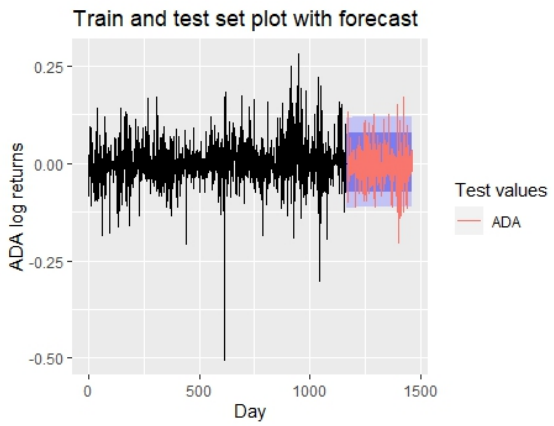
Model	MAPE	RMSE	NRMSE
ARIMA(4,0,2)	1.216911	0.0137081	0.192165

We notice that a 19% NRMSE shows the lack of accuracy for precise prediction. The prediction interval allows us to have an idea of where will the log returns lie with certain probability which is what gives importance to our forecasts.

The results for the ADA model is the following:

Figure 11: **ADA ARIMA model forecasts**

(a) ARIMA(2,0,2): PI: (80,95)

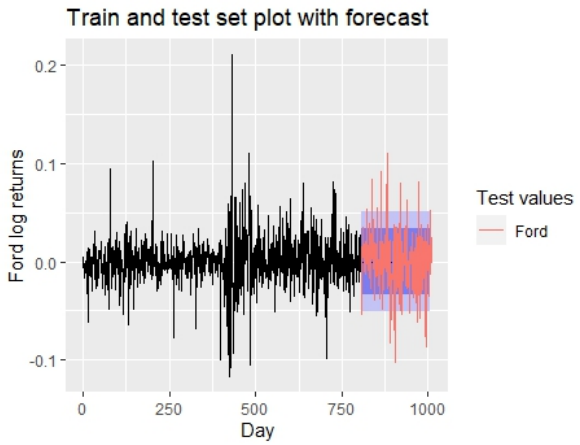


Model	MAPE	RMSE	NRMSE
ARIMA(2,0,2)	1.097022	0.052549	0.140573

The errors produced for the point forecasts in this models do not seem so relevant with a 14% NRMSE error and a 1.09 MAPE. We can conclude that this model is useful thanks to the prediction interval it provides because it indicates where most of the data will lie but when it comes to point forecasts we can see the mean value as the prediction doesn't provide a lot.

Finally we have the forecast plot for the ARIMA(0,0,0) model of the Ford data set. This is basically a white noise process which is entirely random.

Figure 12: **Ford ARIMA model forecast**



Model	RMSE	NRMSE
ARIMA(0,0,0)	0.031200	0.146878

For this model we only got RMSE and NRMSE errors. Even though these errors seem low, especially the 14% NRMSE, we can notice that a lot of values from the test set actually are outside the prediction interval even with a 95% confidence which might indicate that maybe this model is not really good when it comes to predicting the log returns or giving an estimate of where they will lie. This is expected because an ARIMA with those parameters is completely random and it really does not depend on past values of the dependent variable, but we included it to show the similarity and to also fit the optimal ARIMA model to the ford data set coming from the auto.arima function and the Box Jenkins methodology.

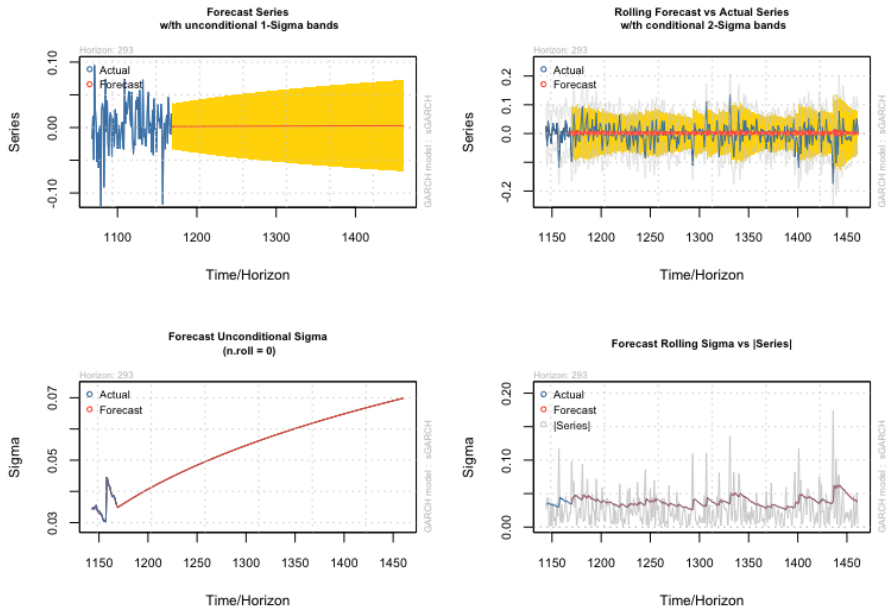
6.2 GARCH models

We will explain and show the results of the GARCH models next. For these results we also used the metrics above to analyze the point forecasts which are the mean of the prediction intervals at every step and we will show how GARCH models, when considering a rolling window, can give more usefulness as the prediction interval is more accurate every time, as it takes recent information into account, since the error variance is not constant. It is important to note that per figure, the two graphs on top are the mean estimates (forecast of the log returns) from the ARMA part of the model,

where the yellow panel is the prediction interval given by the variance forecast either conditional or unconditional. This means it either takes into account the past data with a rolling approach as explained previously or it does not. While the bottom two graphs are the prediction of the conditional and unconditional variance from the GARCH part of the model.

The Bitcoin ARMA(0,2)-GARCH(1,1) forecast is the following:

Figure 13: **Bitcoin GARCH forecasts with conditional and unconditional σ**

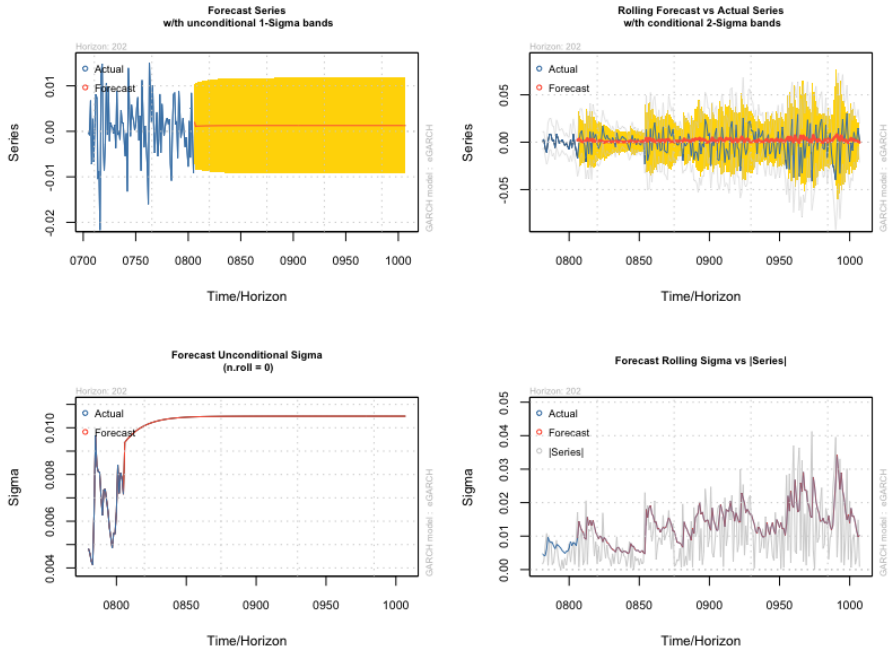


Model	MAPE	RMSE	NRMSE
ARMA(0,2)-GARCH(1,1)	1.049013	0.03669072	0.1184272

We can see from the first plot that the prediction interval increases as the time horizon advances, this is because when we forecast at large horizons the σ prediction is greater as there is more predicted volatility. Also important to note is the information that the rolling window adds to the forecast. We have a more variable prediction of the log returns due to the σ being conditioned on every roll. The results show that the point forecast prediction is near zero as it is the mean of the prediction interval and when using the rolling window approach it varies more around it as the prediction interval is also varying.

The S&P500 ARMA(1,0)-eGARCH(1,1) forecast is the following:

Figure 14: **S&P500 GARCH forecasts with conditional and unconditional σ**

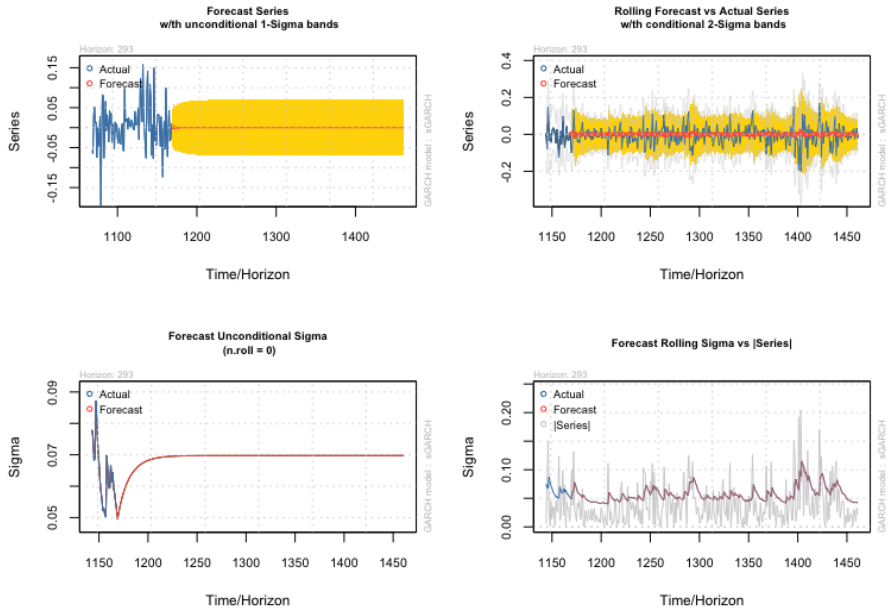


Model	MAPE	RMSE	NRMSE
ARMA(1,0)-eGARCH(1,1)	1.594817	0.01404586	0.1968881

The point forecast errors of this model are higher than the ARIMA model for the S&P500 but since we are able to obtain a prediction interval that varies in accordance with the volatility it gives us a more powerful and useful forecast. We can see on the unconditional forecast that the prediction interval stays almost the same. We can also see on the rolling window approach that the log returns are much more volatile by the end of the time frame.

The ADA ARMA(10,0)-GARCH(1,1) forecast is the following:

Figure 15: **ADA GARCH forecasts with conditional and unconditional σ**

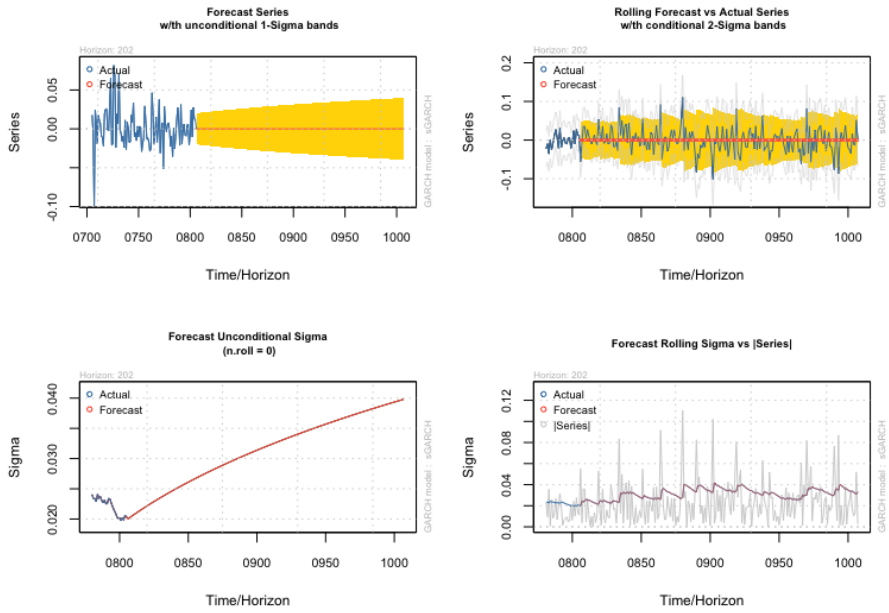


Model	MAPE	RMSE	NRMSE
ARMA(2,2)-GARCH(1,1)	1.157667	0.05262021	0.1407628

Similarly to the last two models, the point forecasts here also produce a worse error than the ARIMA model but it is again important to take into account the uniqueness the GARCH type models add when it comes to prediction intervals. We can see that on the unconditional forecast the value of σ starts small and then converges to a particular number near 0.07. On the rolling window forecast we note that the periods of high volatility have a greater prediction interval which again is more useful from a financial investing analysis perspective as we can make better estimates on the risk involved in the returns of the asset.

The Ford GARCH(1,1) forecast is the following:

Figure 16: **Ford GARCH forecasts with conditional and unconditional** σ



Model	RMSE	NRMSE
GARCH(1,2)	0.03120065	0.1468784

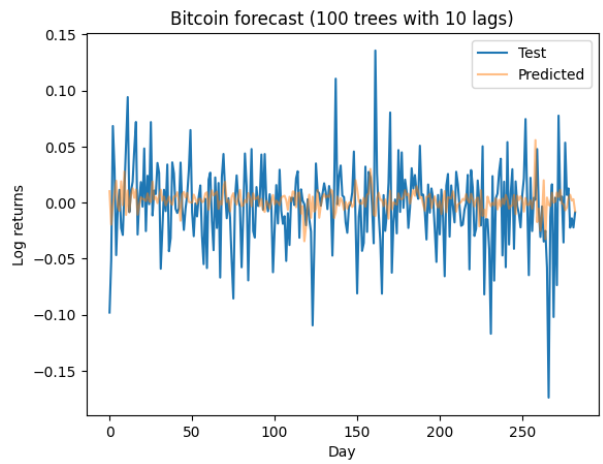
Finally, for the last model the point forecast errors are almost the same (compared to the ARIMA) but we can see that on both graphs the prediction intervals are giving specific information about where the log returns will lie which is what gives the most importance and makes this a more valuable prediction. It is important to note that since this model does not have ARMA terms it starts by assuming the conditional mean to be zero and the point forecasts are therefore all set to zero which does not provide much usefulness as the prediction interval does.

6.3 Random forests models

We will now present the plots of the predictions from the random forest models vs the actual test data for all data sets having used 10 lags of the log returns as the measures in each prediction. After each plot we include the errors obtained.

The bitcoin predictions are the following:

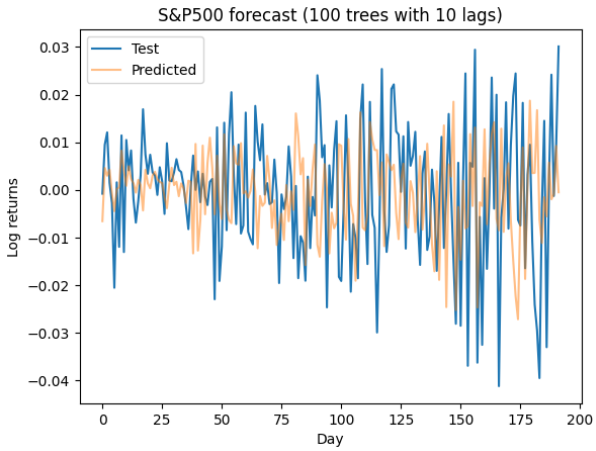
Figure 17: **Bitcoin RF**



RMSE	NRMSE
0.03771	0.1217130

The S&P500 predictions:

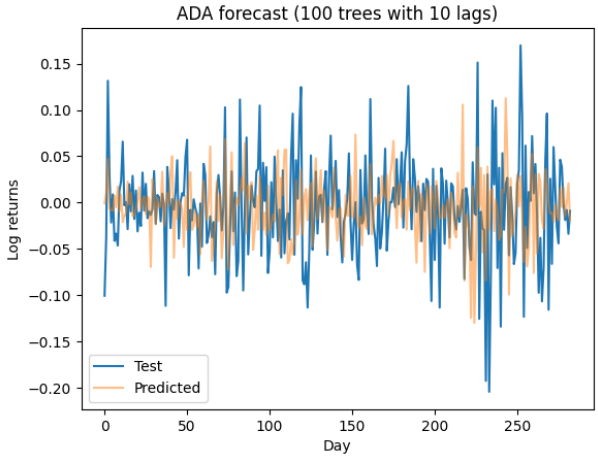
Figure 18: **S&P500 RF**



RMSE	NRMSE
0.00559	0.0783392

The ADA predictions:

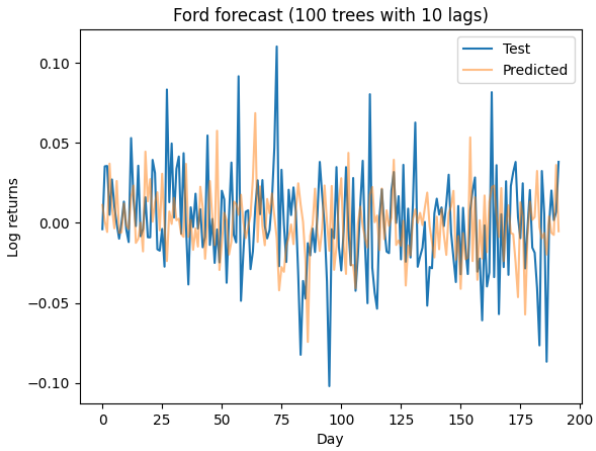
Figure 19: **ADA RF**



RMSE	NRMSE
0.02014	0.0538746

The Ford predictions:

Figure 20: **Ford RF**



RMSE	NRMSE
0.01233	0.0580541

We can see the errors in all models (except for bitcoin) are better in comparison to the previous ARIMA and GARCH point forecasts. It is important to note that these random forests are similar to the GARCH rolling forecast in the sense that in every time step the training set is updated and it allows for one step prediction to happen more effectively. This might be a reason why the predictions are more accurate as they take into consideration a more adequate training set each time, not like the ARIMA forecast that predicts all the test set size in one go. The results show that when it comes to point forecast prediction if we take it as as the mean of the prediction interval from ARIMA and GARCH then random forests appear to be more efficient as they yielded smaller errors. The difference is that random forest models do not provide a prediction interval so they do not really account for the uncertainty in the returns which is something extremely important and valuable that a statistical time series model does provide. It is important to note that we used a 10 day lag for two reasons, the first is that on the GARCH ADA model we noticed a significance with the 10th AR parameter, which led us to believe that this was an important value for the model to recall and the second reason is that 10 lags essentially imply a two week cycle for Ford and S&P500 which are assets that do not trade on weekends as cryptocurrency does.

7 Discussion

It is important to mention again that the error metrics only take the point forecasts into consideration. An improvement for this study could involve analysing the prediction intervals as a whole and obtaining errors based on them. This way we could show the difference in comparison when using ARIMA instead of GARCH as their prediction intervals are indeed better. A future improvement could be done by using bayesian score functions on these intervals. Another thing that could be done to change the perspective of the analysis is instead of obtaining the point forecasts from the mean value, one could obtain it from other statistic measures like the median or a significant quantile. This way the predictions being done could be considered more cautious and less risky. It is also clear that something that would improve the models and their forecasts is adding more variables that might affect the price, as seen in the past literature, this could be volume, interest rates, other asset prices, etc. Another way of improving them would be creating more complex models that allow for time varying parameters. Also, after all the analysis done it was also thought of what could be good financial advice based on the prediction intervals, seeing as the point forecasts are really close to the zero their value alone can be really risky for trying to make profit. If we look at the bottom part of the prediction intervals we can make claims that talk about the risk of investing. For example if we always consider the minimum part of the prediction interval with an 95% confidence we can claim that there is only a 5% chance of our returns going below that value. With this in mind we can show an example for the Bitcoin data. The minimum value of the prediction interval for the best model ARIMA(1,0,1) is -0.07479 then we can guarantee with a 95% probability that the returns will not go below exp(-0.07479) = 0.92793 at any given point from one day to another. This idea could be generalized to do more concise and relevant statements that can talk about the risk of investing, especially if the data is being analyzed on a weekly or a monthly basis as it would be more valuable to know that in a certain week/month we claim that the return won't go below a threshold with a given probability therefore the predictions could be better used for longer term investing. The same analysis could be done with the other time series. Another improvement that could have been done in this study is to not focus on the mean of the predictions (with the ARIMA models) as much, because as we saw, there is more information on the variance due to the type of series we have which pose a lot of sudden spikes and symbolize high volatility. Therefore, having proceeded with the analysis an improvement would have been to give more emphasis to the GARCH models seeing the high volatility the returns pose during the period of time analyzed.

8 Conclusions

Forecasting financial asset's returns is not an easy task, let alone trying to predict them with a high accuracy as it is at times needed and wanted in the economic world. In this study we found important information in two types of statistical time series models applied to asset's returns to use them for forecasting and obtaining accurate prediction intervals. This prediction intervals show us the uncertainty of each forecast. It is clear that the prediction intervals are considerably big on the analyzed models. This makes it awkward to use the forecasts to make accurate investing decisions if the goal of one is to profit but if one wants to mitigate risk and have an idea of where the returns will lie these models can be somewhat useful. We conclude that the point forecasts are not as efficient as it was thought at first (in the statistical models) because they are calculated by taking the mean of the prediction interval (which is always close to zero) and as it has been mentioned they have no significant importance without this interval. The combination of both is what gives more importance to the forecasts made by this type of time series models. We noticed that the prediction intervals with a 95% confidence level are indeed accurate up to a certain point as most of the test data of the assets lie inside of it. This confirms the efficacy of time series models for forecasting where the returns of financial assets will lie. Clearly the ARMA-GARCH models are more important for forecasting returns as they are more general since they allow for the variance of the error terms to vary. This is important to note as the prediction intervals provided with the GARCH forecasts, especially the rolling window, give more precise intervals which allow for a better quantification of where the return will be in the future.

The similarity in all point forecast errors on the time series statistics models made us realize that these methods have a comparable efficiency when it comes to forecasting in one step but it is highly likely that to improve the errors and the accuracy one has to take into account more factors and/or variables apart from only the past returns. Also, really important results were the ones obtained from the random forests as the prediction errors were significantly smaller than on the previous models. We can conclude that for precise one step univariate log return forecasting random forests appear to be superior. It is important to note that random forests work as a type of black box, we do not really have the hypothesis testing and p-values like in the statistical models used but even so we can see the usefulness they can provide.
Finally, with this research we have come to realize that prediction intervals are extremely important in financial forecasting. Improvements could be made by more complex models which take into account either more variables (multivariate) or time

varying parameters which might be able to provide more accurate point forecasts and not only that but smaller prediction intervals, the latter is clearly highly important when it comes to financial investing due to the minimization of uncertainty which implies the more accurate knowledge of the risk involved in an asset's return at a specific point in time.

References

Alahmari, S. A. (2020). Using machine learning arima to predict the price of cryptocurrencies. *The ISC Int'l Journal of information security*, 1-6.

Alessandretti, L., ElBahrawy, A., Aiello, L. M., & Baronchelli, A. (2018). Anticipating cryptocurrency prices using machine learning. *Wiley*, *2018*, 1-16.

Ariyo, A., & Olaniyan, O. (2014). Structural transformation and inequality: Evidence from nigeriaast. *Development*, *57*(3-4), 531-539.

Bianchi, D. (2018). Cryptocurrencies as an asset class: An empirical assessment. *SSRN Electronic Journal*.

Biau, G., & Scornet, E. (2016). A random forest guided tour. *TEST*, *25*, 197-227.

Bollerslev, T., Engle, R. F., & Nelson, D. B. (1994). Arch models. *Handbook of Econometrics*, *4*, 2961-3038.

Box, G. E., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2016). *Volatility estimation for bitcoin: A comparison of garch models*. Wiley.

Breiman, L. (2001). Random forests. *Kluwer Academic Publishers*, *45*, 5-32.

Catania, L., Grassi, S., & Ravazzolo, F. (2018). Forecasting cryptocurrencies financial time series. *Centre for Applied Macro- and Petroleum economics (CAMP), BI Norwegian Business School*, *5*, 1-26.

Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation. *Econometrica*, *50*, 987-1007.

Falessi, D., Huang, J., Narayana, L., & Turhanc, J. F. T. B. (2020). On the need of preserving order of data when validating within-project defect classifiers. *Empir Software Eng*, *25*, 4805-4830.

Finance home. (n.d.). https://finance.yahoo.com/. (Accessed: 2022-07-20)

Fisher, T. J. (2012). Weighted portmanteau tests revisited: Detecting heteroscedasticity, fitting nonlinear and multivariate time series. *SAS Global Forum*, *338*, 1-12.

Granger, C., & Andersen, A. (1978). On the invertibility of time series models. *Stochastic Processes and their Applications*, *8*, 87-92.

Hassani, H., & Yeganegi, M. R. (2019). Selecting optimal lag order in ljung–box tes. *Physica A*, *541*, 1-11.

Hong, Y., & Shehadeh, R. D. (1999). A new test for arch effects and its finite-sample performance. *Journal of Business Economic Statistics*, *17*, 91-108.

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: principles and practice*. OTexts.

Katsiampa, P. (2017). Volatility estimation for bitcoin: A comparison of garch models. *Economics Letters*, *158*, 5.

Knaub, J. R. (2007). Heteroscedasticity and homoscedasticity. *Encyclopedia of measurement and statistics*, 431-432.

Koop, G., & Korobilis, D. (2013). Large time-varying parameter vars. *Journal of Econometrics*, *177*, 185-198.

Learned-Miller, E. G. (2014). Introduction to supervised learning. , 1-5.

Ljung, G., & Box, G. (1978). On a measure of lack of fit in time series models. *Biometrika*, *65*, 297-303.

Ljung, G. M. (1986). Diagnostic testing of univariate time series models. *Biometrika*, *73*, 725-730.

Louppe, G. (2015). *Understanding random forests* (Unpublished doctoral dissertation). University of Liège.

Miller, D., & Kim, J.-M. (2021). Univariate and multivariate machine learning forecasting models on the price returns of cryptocurrencies. *Journal of Risk and Financial Management*, *14*, 1-10.

Miskolczi, P. (2017). Note on simple and logarithmic return. , *11*, 127-136.

Nguyen, K. Q. (2022). The correlation between the stock market and bitcoin during covid-19 and other uncertainty periods. *Finance Research Letters*, *46*, 2-6.

Raftery, A. E., Kárný, M., & Ettler, P. (2010). Online prediction under model uncertainty via dynamic model averaging: Application to a cold rolling mill. *National Institute of Health*, *52*, 1-31.

Ruppert, D. S. M. D. (n.d.). *Garch models of dynamic volatility and correlation.* `https://www.scss.tcd.ie`. (Accessed: 2022-08-20)

Scornet, E. (2020). Trees, forests, and impurity-based variable importance in regression. *Association des Publications de l'Institut Henri Poincaré,*, *59*, 21-52.

Sebastião, H., & Godinho, P. (2021, 01). Forecasting and trading cryptocurrencies with machine learning under changing market conditions. *Financial Innovation*, *7:3*, 1-30. doi: https://doi.org/10.1186/s40854-020-00217-x

Shumway, R. H., & Stoffer, D. S. (2010). *Time series analysis and its applications: With r examples*. Springer.

Statista.com. (2022). *Largest stock exchange operators worldwide as of june 2022, by market capitalization of listed companies.* Retrieved 2022-08-09, from `statista.com`

Stolte, T. (2021). *Predicting cryptocurrency returns in a time-varying volatility context* (Unpublished master's thesis).

Tsay, R. S. (2010). *Analysis of financial time series*. Wiley.

Vo, N., & Slepaczuk, R. (2022). Applying hybrid arima-sgarch in algorithmic investment strategies on sp500 index. *Entropy*, *24*, 1-25.

9 Appendix

9.1 Code 1: Fit random forests

```
# -*- coding: utf-8 -*-
"""RandomForestTS.ipynb

Automatically generated by Colaboratory.

Original file is located at
    https://colab.research.google.com/drive/1rSK1xeSiep7tozqEiyM6S6Qj3c7YKbL0
"""

import pandas as pd
import os
import numpy as np
import statsmodels.tsa.stattools as statsmodels
import matplotlib.pyplot as plt
import math
from sklearn.metrics import mean_squared_error,
    mean_absolute_error, mean_squared_error
from sklearn.ensemble import RandomForestRegressor
from scipy.stats import describe
from pandas_profiling import ProfileReport
from collections import OrderedDict
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier

from google.colab import drive
drive.mount('/content/gdrive')

# Commented out IPython magic to ensure Python compatibility.
# %matplotlib inline

#!pip install pandas-profiling==2.7.1

# Commented out IPython magic to ensure Python compatibility.
# %cd /content/gdrive/MyDrive/tesis/Data

#bitcoin-usd date, open, high, low, close, adj close and
```

```
    volume from july 8 2018 to july 8 2022
df0 = pd.read_csv("BTC-USD.csv")
nTrain = math.floor(df0.shape[0]*0.8) #train two thirds and
    test one third
nTest = df0.shape[0]-nTrain
df0Train = df0[1:nTrain]
df0Test = df0[nTrain:]

obs0 = df0Train.shape[0]
obs0Test = df0Test.shape[0]

dailyLogReturns0Train = df0Train["LogReturns"]

dailyLogReturns0Test = df0Test["LogReturns"]

df1 = pd.read_csv("ADA-USD.csv")
nTrain1 = math.floor(df1.shape[0]*0.8) #train two thirds and
    test one third
nTest1 = df1.shape[0]-nTrain1
df1Train = df1[1:nTrain1]
df1Test = df1[nTrain1:]
dailyLogReturns1Train = df1Train["LogReturns"]

dailyLogReturns1Test = df1Test["LogReturns"]

df2 = pd.read_csv("SP500data.csv")
nTrain2 = math.floor(df2.shape[0]*0.8) #train two thirds and
    test one third
nTest2 = df2.shape[0]-nTrain2
df2Train = df2[1:nTrain2]
df2Test = df2[nTrain2:]
dailyLogReturns2Train = df2Train["LogReturns"]
dailyLogReturns2Test = df2Test["LogReturns"]

df3 = pd.read_csv("F.csv")
nTrain3 = math.floor(df3.shape[0]*0.8) #train two thirds and
    test one third
nTest3 = df3.shape[0]-nTrain3
df3Train = df3[1:nTrain3]
df3Test = df3[nTrain3:]
```

```
dailyLogReturns3Train = df3Train["LogReturns"]
dailyLogReturns3Test = df3Test["LogReturns"]

d = abs(dailyLogReturns0Test.max()-dailyLogReturns0Test.min())
print(dailyLogReturns0Test.max(),dailyLogReturns0Test.min(),
    math.exp(dailyLogReturns0Test.max()))
0.03101/d #% del error entre la distancia total entre los
    returns del test

def MAPE(Y_actual,Y_Predicted):
    mape = np.mean(np.abs((Y_actual -
        Y_Predicted)/Y_actual))*100
    return mape

# transform a time series dataset into a supervised learning
    dataset
def series_to_supervised(data, n_in=1, n_out=1, dropnan=True):
        #n_v = 1 if type(data) is list else data.shape[1]
        df = pd.DataFrame(data)
        cols = list()
        # input sequence (t-n, ... t-1)
        for i in range(n_in, 0, -1):
                cols.append(df.shift(i))
        # forecast sequence (t, t+1, ... t+n)
        for i in range(0, n_out):
                cols.append(df.shift(-i))
        # put it all together
        agg = pd.concat(cols, axis=1)
        # drop rows with NaN values
        if dropnan:
                agg.dropna(inplace=True)
        return agg.values

sTrain = series_to_supervised(dailyLogReturns0Train,n_in=7)
    #turns time series into supervised data learning problem
#aqui solo estoy usando un parametro que son los log returns
    pero podrian ser mas
sTest = series_to_supervised(dailyLogReturns0Test,n_in=7)
#n_in es como los lags osea cuantos mas observaciones futuras
    se incluyen en cada lista
```

```
sTrain1 = series_to_supervised(dailyLogReturns1Test,n_in=7)
sTest1 = series_to_supervised(dailyLogReturns1Test,n_in=7)

sTrain2 = series_to_supervised(dailyLogReturns2Test,n_in=7)
sTest2 = series_to_supervised(dailyLogReturns2Test,n_in=7)

sTrain3 = series_to_supervised(dailyLogReturns3Test,n_in=7)
sTest3 = series_to_supervised(dailyLogReturns3Test,n_in=7)

#walk forward validation porque k fold cross validation no
    puede ser porque no nos serivira que la data este
#random necesitamos que la data siga el orden de la ts, el
    pasado predice en el futuro y no viceversa
#Nota de la pagina importante:If we are interested in making a
    one-step forecast, e.g. one month, then we can evaluate the
    model by training on the training dataset and predicting
    the first step in the test dataset. We can then add the
    real observation from the test set to the training dataset,
    refit the model, then have the model predict the second
    step in the test dataset.
#citar este codigo preguntar a jackie

# walk-forward validation for univariate data
def walk_forward_validation(train,test): #si meto trian y test
    en vez de data es menos general pero ya los habia dividido
        predictions = list()
        # split dataset
        #train, test = train_test_split(data, n_test)
        # seed history with training dataset
        history = [x for x in train]
        # step over each time-step in the test set
        for i in range(len(test)):
                # split test row into input and output columns
                testX, testy = test[i, :-1], test[i, -1]
                # fit model on history and make a prediction
                yhat = random_forest_forecast(history, testX)
                # store forecast in list of predictions
                predictions.append(yhat)
                # add actual observation to history for the
```

```
                    next loop
                history.append(test[i])
                # summarize progress
                print('>expected=%.5f, predicted=%.5f' %
                    (testy, yhat))
        # estimate prediction error
        mae = mean_absolute_error(test[:, -1], predictions)
        rmse =
            math.sqrt(mean_squared_error(test[:,-1],predictions))
        return mae, rmse, test[:, 1], predictions,

def random_forest_forecast(train, testX):
        # transform list into array
        train = np.asarray(train)
        # split into input and output columns
        trainX, trainy = train[:, :-1], train[:, -1]
        # fit model
        model =
            RandomForestRegressor(n_estimators=100,max_features=2)
            #using 7/3 features which rounding is 2 because
            that is why Breiman recommends for regression
            problems.
        model.fit(trainX, trainy)
        # make a one-step prediction
        yhat = model.predict([testX])
        return yhat[0]

mae, rmse, a, b = walk_forward_validation(sTrain,sTest) #btc
    100 arboles n_estimators #7 lags

mae1, rmse1, a1, b1 = walk_forward_validation(sTrain1,sTest1)
    #ada

mae2, rmse2, a2, b2 = walk_forward_validation(sTrain2,sTest2)
    #sp500

mae3, rmse3, a3, b3 = walk_forward_validation(sTrain3,sTest3)
    #ford

d0 = abs(dailyLogReturns0Test.max()-dailyLogReturns0Test.min())
```

```
d1 = abs(dailyLogReturns1Test.max()-dailyLogReturns1Test.min())
d2 = abs(dailyLogReturns2Test.max()-dailyLogReturns2Test.min())
d3 = abs(dailyLogReturns3Test.max()-dailyLogReturns3Test.min())

plt.plot(a,label="Test")
plt.plot(b,label="Predicted",alpha=0.5)
plt.legend()
plt.title("Bitcoin forecast (100 trees with 7 lags)")
plt.xlabel("Day")
plt.ylabel("Log returns")
plt.show()
print("MAE=%.5f and RMSE=%.5f" %(mae,rmse),MAPE(a,b), rmse/d0)
#plt.savefig('btc.jpeg')

plt.plot(a1,label="Test")
plt.plot(b1,label="Predicted",alpha=0.5)
plt.legend()
plt.title("ADA forecast (100 trees with 7 lags)")
plt.xlabel("Day")
plt.ylabel("Log returns")
plt.show()
print("MAE=%.5f and RMSE=%.5f" %(mae1,rmse1), MAPE(a1,b1),
    rmse1/d1)
#plt.savefig('ada.png')

plt.plot(a2,label="Test")
plt.plot(b2,label="Predicted",alpha=0.5)
plt.legend()
plt.title("S&P500 forecast (100 trees with 7 lags)")
plt.xlabel("Day")
plt.ylabel("Log returns")
plt.show()
print("MAE=%.5f and RMSE=%.5f" %(mae2,rmse2), MAPE(a2,b2),
    rmse2/d2)
#plt.savefig('sp.png')

plt.plot(a3,label="Test")
plt.plot(b3,label="Predicted",alpha=0.5)
plt.legend()
plt.title("Ford forecast (100 trees with 7 lags)")
```

```
plt.xlabel("Day")
plt.ylabel("Log returns")
plt.show()
print("MAE=%.5f and RMSE=%.5f" %(mae3,rmse3),
    MAPE(a3,b3),rmse3/d3)
#plt.savefig('f.png')
```

9.2 Code 2: Fit univariate time series GARCH

```
library(rugarch) # GARCH models
library(forecast) # checkresiduals
library(ggplot2) # residual plots


#Data read (obtained from yahoo finance and bloomberg):

df0 = read.csv("BTC-USD.csv")
nTrain = floor(dim(df0)[1]*.8) #80% for train set
nTest = dim(df0)[1] - nTrain
bTS =
    ts(as.numeric(as.character(df0$LogReturns[2:dim(df0)[1]])))
# n closing prices => n - 1 returns

df1 = read.csv("SP500data.csv")
nTrain1 = floor(dim(df1)[1]*.8) #80% for train set
nTest1 = dim(df1)[1]-nTrain1 #20% test set
sTS =
    ts(as.numeric(as.character(df1$LogReturns[2:dim(df1)[1]])))

df2 = read.csv("ADA-USD.csv") #same size as bitcoin so nTrain
    and nTest are the
#same no need to compute again
aTS =
    ts(as.numeric(as.character(df2$LogReturns[2:dim(df2)[1]])))

df3 = read.csv("F.csv")
fTS =
    ts(as.numeric(as.character(df3$LogReturns[2:dim(df3)[1]])))


#Function to iterate across different garch fits and save aic
    scores

garchfit = function(p, q, npg, nqg, data,outsample){
  d = c() #Dictionary where the keys will be AIC scores and
      the values order.
  for (n in 0:p){
```

```
    for (m in 0:q){
      for (i in 1:npg){ #First parameter of Garch is 1 for there to be a GARCH
        for (j in 0:nqg){
          garch.spec = ugarchspec(variance.model = list(garchOrder=c(i,j)),
                                  mean.model = list(armaOrder=c(n,m),
                                                    include.mean = TRUE,
                                                    archm=TRUE),
                                  distribution.model = "std")
          #We always use student t as it shows to have a better AIC
          #when fitting this type of GARCH models.
          try({garch.fit = ugarchfit(spec=garch.spec, data=data,
                                     solver.control=list(trace = 1),
                                     out.sample=outsample)})
          #Try because some model orders fail to converge and to avoid crashing
          d[as.character(infocriteria(garch.fit)[1])] = list(c(n,m,i,j))
          print(cat(n,m,i,j))

          }

        }
      }

    }
  return(d)
}

AB = garchfit(2, 2, 3, 3, bTS,nTest)
l = c()
for (score in names(AB)){
  s = as.numeric(score)
  l = append(l,s)
}
```

```
k = min(l)
#The minimum AIC model has order:
AB[as.character(k)] # 0 2 1 1
#The smallest AIC scored is obtained with the model ARMA(0,2)-GARCH(1,1)
#with the a_t following a student t distribution.

#Bitcoin fit:
garch.specB = ugarchspec(variance.model = list(garchOrder=c(1,1)),
                         mean.model = list(armaOrder=c(0,2),
                                           include.mean=FALSE,archm=TRUE),
                         distribution.model = "std")

garch.fitB = ugarchfit(spec=garch.specB, data=bTS,
                       solver.control=list(trace = 1),out.sample=nTest)
plot(garch.fitB,which="all")

#Bitcoin forecast:
bf = ugarchforecast(garch.fitB,n.ahead=nTest,n.roll=nTest)

#S&P500:
AS = garchfit(4, 2, 3, 3, sTS,nTest1)
l = c()
for (score in names(AS)){
  s = as.numeric(score)
  l = append(l,s)
}
k = min(l)
#El minimo AIC tiene order:
AS[as.character(k)] # 1 0 1 1
#The smallest AIC scored is obtained with the model ARMA(1,0)-GARCH(1,1)
#with the a_t following a student t distribution.
#S&P500 fit:
garch.specS = ugarchspec(variance.model = list(model = "eGARCH",
                                               garchOrder=c(1,1)),
                         mean.model = list(armaOrder=c(1,0),
```

```
                                                include.mean =
                                                    FALSE,archm=TRUE),
                            distribution.model = "norm")

garch.fitS = ugarchfit(spec=garch.specS, data=sTS,
                        solver.control=list(trace =
                            1),out.sample=nTest1)
plot(garch.fitS,which="all")


#S&P500 forecast:
sf = ugarchforecast(garch.fitS,n.ahead=nTest1,n.roll=nTest1)

#ADA:
AA = garchfit(2, 2, 3, 3, aTS,nTest)
l = c()
for (score in names(AA)){
  s = as.numeric(score)
  l = append(l,s)
}
k = min(l)
#El minimo AIC tiene order:
AA[as.character(k)] # 2 2 2 1
#The smallest AIC scored is obtained with the model
    ARMA(2,2)-GARCH(2,1)
#with the a_t following a student t distribution.

#ADA fit:
#Model 1
garch.specA = ugarchspec(variance.model =
    list(garchOrder=c(1,1)),
                            mean.model = list(armaOrder=c(10,0),
                                                include.mean=FALSE),
                            distribution.model = "std",
                            fixed.pars =
                                list("ar2"=0,"ar3"=0,"ar4"=0,"ar5"=0,
                                                "ar6"=0,"ar7"=0,"ar8"=0,"ar9"=0))

garch.fitA = ugarchfit(spec=garch.specA, data=aTS,
                        solver.control=list(trace =
```

```
                                1),out.sample=nTest)
plot(garch.fitA2,which="all")

#Model 2
garch.specA2 = ugarchspec(variance.model =
    list(garchOrder=c(1,1)),
                            mean.model =
                                list(armaOrder=c(1,0),include.mean=FALSE),
                            distribution.model = "std")
garch.fitA2 = ugarchfit(spec=garch.specA2, data=aTS,
                        solver.control=list(trace =
                                1),out.sample=nTest)

#ADA forecast:
af = ugarchforecast(garch.fitA,n.ahead=nTest,n.roll=nTest)

#Ford:

AF = garchfit(0, 0, 3, 3, fTS,nTest1)
l = c()
for (score in names(AF)){
  s = as.numeric(score)
  l = append(l,s)
}
k = min(l)
#El minimo aic tiene order:
AF[as.character(k)] # 0 0 1 2

#Ford fit:
garch.specF = ugarchspec(variance.model =
    list(garchOrder=c(1,1)),
                            mean.model = list(armaOrder=c(0,0),
                                              include.mean =
                                                  FALSE,archm=FALSE),
                            distribution.model = "std")

garch.fitF = ugarchfit(spec=garch.specF, data=fTS,
                        solver.control=list(trace =
                                1),out.sample=nTest1)
plot(garch.fitF,which="all")
```

```
#Ford forecast:
ff = ugarchforecast(garch.fitF,n.ahead=nTest1,n.roll=nTest1)

#Functions for errors:
mimape = function(ypred,ytest){
  s = 0
  c=1
  for (i in ypred){
    s = s + abs((ytest[c]-i)/ytest[c])
    c = c + 1
  }
  mape = s/(length(ypred))
  return(mape)
}

mirmse = function(ypred,ytest){
  s = 0
  c=1
  for (i in ypred){
    s = s + (ytest[c]-i)^2
    c = c + 1
  }
  rmse = sqrt(s/(length(ypred)))
  return(rmse)
}

#possiblity of dividing over mean or median not only size
mirelrmse = function(ypred,ytest){
  s = 0
  c=1
  for (i in ypred){
    s = s + (ytest[c]-i)^2
    c = c + 1
  }
  rmse = sqrt(s/(length(ypred)))/(max(ytest)-min(ytest))
  return(rmse)
}

#Preparamos data para obtener ultimas entradas
```

```
    (correspondientes al test set)

dlrTestb = tail(bTS,293)
dlrTests = tail(sTS,202)
dlrTesta = tail(aTS,293)
dlrTestf = tail(fTS,202)

#Error calculations:

bpred = fitted(bf)[1,1:293]
spred = fitted(sf)[1,1:202]
apred = fitted(af)[1,1:293]
fpred = fitted(ff)[1,1:202]

mapeb = mimape(bpred,dlrTestb)
mapes = mimape(spred,dlrTests)
mapea = mimape(apred,dlrTesta)
mapef = mimape(fpred,dlrTestf)

rmseb = mirmse(bpred,dlrTestb)
rmses = mirmse(spred,dlrTests)
rmsea = mirmse(apred,dlrTesta)
rmsef = mirmse(fpred,dlrTestf)

nrmseb = mirelrmse(bpred,dlrTestb)
nrmses = mirelrmse(spred,dlrTests)
nrmsea = mirelrmse(apred,dlrTesta)
nrmsef = mirelrmse(fpred,dlrTestf)
```

9.3 Code 3: Fit univariate time series ARIMA

```
library(forecast)
library(MLmetrics)
library(tseries)
library(ggplot2)
library(psych)
library(stats)
library(bamlss)
library(verification)


#errors
mimape = function(ypred,ytest){
  s = 0
  c=1
  for (i in ypred){
    s = s + abs((ytest[c]-i)/ytest[c])
    c = c + 1
  }
  mape = s/(length(ypred))
  return(mape)
}

mirmse = function(ypred,ytest){
  s = 0
  c=1
  for (i in ypred){
    s = s + (ytest[c]-i)^2
    c = c + 1
  }
  rmse = sqrt(s/(length(ypred)))
  return(rmse)
}

#rel rmse dividiendo entre el rango del test
#possiblity of dividing over mean or median not only size
mirelrmse = function(ypred,ytest){
  s = 0
  c=1
```

```
  for (i in ypred){
    s = s + (ytest[c]-i)^2
    c = c + 1
  }
  rmse = sqrt(s/(length(ypred)))/(max(ytest)-min(ytest))
  return(rmse)
}


#Data read (obtained from yahoo finance and bloomberg)

#BTC
#bitcoin-usd date, open, high, low, close, adj close,
    volume,Returns (daily),
# LogReturns from July 8 2018 to July 8 2022 data frame

df0 = read.csv("BTC-USD.csv")
nTrain0 = floor(dim(df0)[1]*.8) #80% for train set
nTest0 = dim(df0)[1]-nTrain0 #20% test set
df0$LogReturns = as.numeric(as.character(df0$LogReturns))
a = nTrain0+1 #Couldn t make the below line work as desired
    without this.
dlrTrainb = ts(df0[2:nTrain0,9])
#Empezamos en el dos porque el primer log return es 0 porque
    no hay return t-1
dlrTestb = ts(df0[a:(dim(df0)[1]),9],start=a,frequency=1)
#Remove one from dimension because first test set value is zero

#SP500
#Same data file information as btc and same dates
#(important to consider that sp500)
# does not have prices during the weekend as the market is
    closed
#and also on certain holidays

df1 = read.csv("SP500data.csv")
nTrain1 = floor(dim(df1)[1]*.8) #80% for train set
nTest1 = dim(df1)[1]-nTrain1 #20% test set
df1$LogReturns = as.numeric(as.character(df1$LogReturns))
a = nTrain1+1
```

```
dlrTrains = ts(df1[2:nTrain1,7])
dlrTests = ts(df1[a:(dim(df1)[1]),7],start=a,frequency=1)

#ADA
df2 = read.csv("ADA-USD.csv") #same data file information as
    btc and same dates
nTrain2 = floor(dim(df2)[1]*.8) #80% for train set
nTest2 = dim(df2)[1]-nTrain2 #20% test set
df2$LogReturns = as.numeric(as.character(df2$LogReturns))
a = nTrain2+1
dlrTraina = ts(df2[2:nTrain2,9])
dlrTesta = ts(df2[a:(dim(df2)[1]),9],start=a,frequency=1)

#Ford
#same data file information as btc and same dates but taking
    into account that
#like sp500 it doesn't include weekends and close days
#Note: sp500 file doesnt have the same data as others because
    it comes from
#bloomberg and not yahoo finance

df3 = read.csv("F.csv")
nTrain3 = floor(dim(df3)[1]*.8) #80% for train set
nTest3 = dim(df3)[1]-nTrain3 #20% test set
df3$LogReturns = as.numeric(as.character(df3$LogReturns))
a = nTrain3+1
dlrTrainf = ts(df3[2:nTrain3,9])
dlrTestf = ts(df3[a:(dim(df3)[1]),9],start=a,frequency=1)

#BTC
fit0 =
    auto.arima(dlrTrainb,stepwise=FALSE,approximation=FALSE,max.p
    = 10,
                    max.q = 10, max.d = 3,trace=TRUE) #arima(101)
fit1 = arima(dlrTrainb, order = c(2,0,2))
y1 = forecast(fit0,h=length(dlrTestb)) #forecasting with both
y2 = forecast(fit1, h=length(dlrTestb))

#residual analysis
Box.test(fit0$residuals,lag=1,type="Ljung-Box")
```

```
checkresiduals(fit0$residuals,df=1)
Bitcoin = dlrTestb
b1 = autoplot(y1)+autolayer(Bitcoin)
b1[9]$labels$y = "Bitcoin daily log returns"
b1[9]$labels$x = "Day"
b1[9]$labels$title = "Train and test set plot with forecast"
b1[9]$labels$colour = "Test values"
cat("MAPE, RMSE and NRMSE for ARIMA(1,0,1)",
    mimape(y1$mean,dlrTestb),
    mirmse(y1$mean,dlrTestb),mirelrmse(y1$mean,dlrTestb))
b1 #plot fit0


b2 = autoplot(y2)+autolayer(Bitcoin)
b2[9]$labels$y = "Bitcoin daily log returns"
b2[9]$labels$x = "Day"
b2[9]$labels$title = "Train and test set plot with forecast"
b2[9]$labels$colour = "Test values"
cat("MAPE, RMSE and NRMSE for ARIMA(2,0,2)",
    mimape(y2$mean,dlrTestb),
    mirmse(y2$mean,dlrTestb),mirelrmse(y2$mean,dlrTestb))
b2 #plot fit1


#sp500
fit2 =
    auto.arima(dlrTrains,stepwise=FALSE,approximation=FALSE,max.p
    = 10,
                    max.q = 10, max.d = 3) #best model is
                        arima(4,0,1)
pacf(dlrTrains)
acf(dlrTrains)

#Analyzing pacf and acf, another possible good option:
fit3 = arima(dlrTrains, order = c(4,0,2))

y3 = forecast(fit2,h=length(dlrTests))
y4 = forecast(fit3, h=length(dlrTests))

b4 = autoplot(y4)+autolayer(SP500)
```

```
b4[9]$labels$y = "S&P500 daily log returns"
b4[9]$labels$x = "Day"
b4[9]$labels$title = "Train and test set plot with forecast"
b4[9]$labels$colour = "Test values"
cat("MAPE, RMSE and NRMSE for ARIMA(4,0,2)",
    mimape(y4$mean,dlrTests),
    mirmse(y4$mean,dlrTests),mirelrmse(y4$mean,dlrTests))
b4

#ada
fit6 =
    auto.arima(dlrTraina,stepwise=FALSE,approximation=FALSE,max.p
    = 10,
                    max.q = 10, max.d = 3) #best model is
                        arima(5,1,0)
#now we are going to use box jenkins to manually find another
    good fit
pacf(dlrTraina) #first two lags outside so ar2 seems a good fit
acf(dlrTraina) #first two lags outside so ma2
fit7 = arima(dlrTraina, order = c(2,0,2))
tsdiag(fit7)
tsdiag(fit6)


b6 = autoplot(y6)+autolayer(ADA)
b6[9]$labels$y = "ADA log returns"
b6[9]$labels$x = "Day"
b6[9]$labels$title = "Train and test set plot with forecast"
b6[9]$labels$colour = "Test values"
cat("MAPE, RMSE and NRMSE for ARIMA(2,0,2)",
    mimape(y6$mean,dlrTesta),
    mirmse(y6$mean,dlrTesta),mirelrmse(y6$mean,dlrTesta))
b6

#ford
fit8 =
    auto.arima(dlrTrainf,stepwise=FALSE,approximation=FALSE,max.p
    = 10,
                    max.q = 10, max.d = 3) #best model is
                        arima(0,0,0)
```

```
y7 = forecast(fit8,length(dlrTestf))
Ford = dlrTestf
b7 = autoplot(y7)+autolayer(Ford)
b7[9]$labels$y = "Ford log returns"
b7[9]$labels$x = "Day"
b7[9]$labels$title = "Train and test set plot with forecast"
b7[9]$labels$colour = "Test values"
cat("MAPE, RMSE and NRMSE for ARIMA(0,0,0)",
    mimape(y7$mean,dlrTestf),
    mirmse(y7$mean,dlrTestf),mirelrmse(y7$mean,dlrTestf))
b7




#data description
btc = describe(df0$LogReturns)
sp500 = describe(df1$LogReturns)
ada = describe(df2$LogReturns)
f = describe(df3$LogReturns)

btctr = describe(dlrTrainb)
btcte = describe(dlrTestb)

sp500tr = describe(dlrTrains)
sp500te = describe(dlrTests)

adatr = describe(dlrTraina)
adate = describe(dlrTesta)

ftr = describe(dlrTrainf)
fte = describe(dlrTestf)

#Train set graphs
par(mfrow=c(2,2))
plot(dlrTrainb,main="Bitcoin train set",ylab="Log
    returns",xlab="Day")
plot(dlrTrains,main="S$P500 train set",ylab="Log
    returns",xlab="Day")
plot(dlrTraina,main="ADA train set",ylab="Log
```

```
    returns",xlab="Day")
plot(dlrTrainf,main="Ford train set",ylab="Log
    returns",xlab="Day")


#Ljung-Box:
residualAnalysis = function(fits,alpha){
  if (alpha==0.05){
    h=50
  }else{
    h=25
  }
  for (i in fits){
    s = arimaorder(i)[1]+arimaorder(i)[3]
    print(Box.test(i$residuals,lag=h,type="Ljung-Box",fitdf=s))
  }
}
fits = list(fit0,fit1,fit6,fit7,fit2,fit3,fit8)
residualAnalysis(fits,0.05) #alpha 5% corresponds to 50 lags
residualAnalysis(fits) #alpha 10% corresponds to 25
```